

Override-Locked Decisions in Hybrid NIDS: Formalization and Counterfactual Decomposition

Mohamed Benjari^a

^a*School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing, China*

Abstract

Anomaly-based network intrusion detection systems (NIDS) achieve high detection rates but produce decisions that security analysts cannot interpret, verify, or act upon. This paper identifies a structurally distinct class of enforcement decisions — *override-locked decisions* — in hybrid NIDS that combine anomaly detection with rule-based classification and policy-level overrides. We formalize this class, develop a three-class taxonomy of enforcement decisions, and present a multi-pass counterfactual engine with a dedicated mechanism for the override-locked case. Any counterfactual method restricted to continuous-feature perturbation is structurally bounded to a coverage ceiling of 10.47% on the evaluated capture; the proposed engine achieves 97.61% coverage in real-time mode and 100% in the offline configuration, at a mean latency of 0.06 ms per packet. The remaining 2.39% shortfall is structural rather than latency-driven, arising from a decision class not covered by the real-time path. We also document a feedback learning subsystem and disclose a vulnerability to sustained adversarial feedback. These results are obtained on a single Mirai botnet capture; coverage figures do not generalise to captures with different attack distributions, and detection labels are self-produced without external ground truth.

Keywords: network intrusion detection, counterfactual explanation, hybrid NIDS, override-locked decisions, explainable AI, feedback learning, anomaly detection, KitNET, decision taxonomy, real-time explanation

1. Introduction

The volume and sophistication of attacks targeting modern networks have made automated enforcement an operational necessity: human analysts cannot triage, in real time, the alert stream produced by a large-scale deployment without automated decision support. Network intrusion detection systems (NIDS) have responded to this pressure by evolving into hybrid architectures that couple a continuous anomaly detector with a rule-based classifier and a structured decision layer. The anomaly detector quantifies how unusual the observed traffic is; the classifier assigns a semantic attack label from a finite label space; and the decision layer maps these inputs to a discrete enforcement action that determines how the network responds. In production, the decision layer typically also implements one or more *policy overrides* that force a specific action whenever the classifier assigns a label of operational concern, regardless of the underlying continuous signal. This compositional pattern — anomaly score, semantic label, and policy override — is the prevalent architectural pattern in modern hybrid NIDS [1, 2, 3].

As hybrid NIDS have been deployed more widely, the interpretability of their decisions has become an operational requirement in practice. Security operations centers process thousands of alerts per shift,

Email address: benjari2004@gmail.com (Mohamed Benjari)

and analysts triage them under time pressure [4, 5]. An enforcement action that cannot be explained in concrete, actionable terms impedes verification, audit accountability, and remediation [3, 6]. Explainable AI (XAI) methods originally developed for tabular classification have therefore been adapted to the intrusion detection setting. Feature-attribution methods — including local surrogates such as LIME [7] and additive attributions such as SHAP [8] — explain which inputs drove a particular model output. Counterfactual explanation methods — including those of Wachter et al. [9], DICE [10], and LEMNA [11] — identify the minimal change to the input that would have altered the decision. The two families answer complementary questions, and neither subsumes the other. This paper is in the counterfactual family. The comparison with attribution baselines in Section 5.5 is scoped to the counterfactual coverage question, because LIME and SHAP address a complementary but distinct explanatory goal.

1.1. Problem Statement

Standard counterfactual explanation methods rest on a shared structural assumption: the decision under explanation depends, at least in some neighbourhood of the query point, on the continuous features used as the perturbation search space. For purely score-driven detectors, where enforcement severity increases monotonically with the anomaly score, this assumption is satisfied and counterfactual search is well-founded. It breaks down, however, for the class of decisions in which a categorical policy override fully determines the enforcement action, independently of any continuous signal.

To see why, consider a hybrid NIDS that issues a BLOCK action whenever the classifier assigns the label *Mirai C2 Communication*. No perturbation of the anomaly score, the risk score, or the per-source contact frequency can change that decision: the override matrix forces BLOCK irrespective of those values. A counterfactual search over the continuous feature space terminates without a valid result — not because the search budget is insufficient, but because the variable whose change would alter the action lies entirely outside the perturbation space. We call decisions of this kind *override-locked decisions*. They are formalized in Definition 1 and situated within a broader taxonomy of enforcement decisions in Section 4.2.

1.2. Research Gap

The counterfactual explanation literature has not, to our knowledge, treated the override-locked class as a structurally distinct case requiring its own mechanism. Surveys of interpretable machine learning [12] frame the explanation problem as a search over a continuous or mixed feature space. The model-agnostic framework of Karimi et al. [13] extends counterfactual search to consequential decisions with mixed feature types and structural perturbation constraints, but does not address the case in which a categorical variable, by virtue of an external policy, unilaterally fixes the action regardless of all continuous inputs.

Explanation methods applied to anomaly detectors and rule-based classifiers have similarly not reached this case. Antwarg et al. [14] and Wang et al. [15] apply SHAP and LIME to explain which features contribute to a high anomaly score — a valuable capability, but one that targets the detection layer, not the downstream enforcement action. The most closely related recent work is xNIDS [16], which explains deep-learning NIDS decisions by approximating history inputs and capturing feature dependencies to support active intrusion response; it represents the current state of the art in explanation fidelity for learning-based detectors. Yet xNIDS, like its predecessors, targets the detection-layer output. When a categorical policy override supersedes that output and determines the final enforcement action independently of any learned score, the resulting decision falls outside the explanatory scope of all existing methods.

In the evaluation reported in Section 5, the majority of detected anomalies are override-locked, and any counterfactual method restricted to perturbing the continuous detection features — regardless of whether the attack-type label is included in a post-hoc surrogate — is structurally limited to explaining only a small minority of decisions. Section 5.5 quantifies this gap and evaluates the proposed engine against it.

1.3. Research Questions

This paper addresses five questions. The first concerns formalization: how can the class of decisions that are structurally inaccessible to continuous-feature counterfactual methods be precisely characterized, and how does it relate to the other decision classes that arise in a hybrid NIDS? The second concerns mechanism:

what explanation procedure produces a valid counterfactual for an override-locked decision, and how does it compose with mechanisms suited to other classes? The third concerns empirical coverage: how does the proposed engine compare, on the same anomaly set, against existing continuous-feature baselines? The fourth concerns latency: what per-packet processing overhead does the mechanism impose, and is this overhead compatible with real-time deployment? The fifth concerns adaptation: under analyst feedback, does the system adapt the detection boundary in the expected direction, and under what conditions does the drift signal fail to fire?

1.4. Contributions

This paper makes four contributions.

1. **Formalization of override-locked decisions and a structural coverage bound.** Definition 1 and the invariance property of Equation 3 formalize the class of enforcement decisions rendered invariant under continuous-feature perturbation by a categorical policy override. From the three-class taxonomy of Section 4.2, we derive the analytic coverage ceiling for any method restricted to the continuous perturbation space on the evaluated capture and confirm it empirically via grid search (Section 5.5).
2. **Taxonomy of enforcement decisions in hybrid NIDS.** A partition of the anomaly set into Class A (continuously-explainable), Class B (threshold-locked), and Class C (override-locked), with a formal proof of exhaustiveness and mutual exclusivity (Remark 1). The taxonomy makes explicit which explanation mechanisms are applicable to which decision class — a distinction absent from the prior counterfactual literature — and determines which pass the engine invokes for a given anomaly.
3. **Override decomposition algorithm and multi-pass counterfactual engine.** Algorithm 2 resolves Class C decisions by stripping the categorical override, re-evaluating the decision function under the resulting baseline policy, and searching the continuous-feature grid against the baseline action. Composed with Pass 1 (single-feature, Class A), Pass 3 (temporal, supplementary), and Pass 4 (multi-feature, Class B, offline only), the engine achieves complete coverage on the evaluated Mirai capture in offline mode and near-complete coverage at sub-millisecond mean latency with zero timeouts in real-time mode across four budget configurations. To our knowledge, Algorithm 2 is among the first counterfactual formulations explicitly designed for categorical override mechanisms in hybrid NIDS (Section 1.2). Coverage figures are specific to this capture’s class distribution; Section 5.10 discloses generalizability limits explicitly.
4. **Feedback learning subsystem with explicit poisoning failure mode.** A bounded-step threshold adaptation and multiplicative weight-update mechanism adapts the detection boundary under analyst feedback. We formally characterize a poisoning failure mode in which a sustained adversarial feedback pattern prevents the drift detector from firing, trace it to the baseline-relative formulation of the trigger condition (Section 5.9.4), and relate it to the class of vulnerabilities identified by Apruzzese et al. [6].

1.5. Paper Organization

Section 2 surveys related work across the three strands of literature that inform this paper: counterfactual explanation, explainability for intrusion detection, and feedback learning and drift detection. Section 3 presents the framework architecture. Section 4 develops the formalization, taxonomy, and technical specification of each component. Section 5 reports the experimental evaluation, including baseline coverage analysis (Section 5.5) and quantitative ablation (Section 5.8), before Section 6 draws conclusions and identifies directions for future work.

2. Related Work

This section positions the present contribution against three related strands of literature: counterfactual and post hoc explanation methods (Section 2.1), explainability methods applied to anomaly detection and intrusion detection (Section 2.2), and adaptation mechanisms based on drift detection and analyst feedback (Section 2.3). Section 2.4 synthesizes the discussion into an explicit statement of the research gap.

2.1. Counterfactual and Post-Hoc Explanation Methods

2.1.1. Counterfactual Explanations

The counterfactual formulation of model explanation was articulated by Wachter et al. [9] in the context of automated decision-making under the GDPR. DICE [10] introduced diverse counterfactuals to address the insufficiency of a single nearest counterfactual for characterizing the decision boundary in actionable terms. LEMNA [11] adapted local surrogate ideas to deep learning security models, with attention to feature dependencies that linear surrogates fail to capture. The model-agnostic formulation of Karimi et al. [13] broadened the framework to consequential decisions with mixed feature types and structural perturbation constraints.

Each of these methods operates by perturbing a continuous (or mixed) feature vector and searching for a configuration that changes the model output. None explicitly addresses the case in which the decision is fully determined by a categorical variable set by an external policy — formalized as Definition 1 in this paper. As a consequence, these methods cannot produce a valid counterfactual for any Class C decision (Section 4.2.3), regardless of search budget or implementation quality. A comprehensive review of over 350 counterfactual and algorithmic-recourse methods — including their proximity, plausibility, and actionability properties — is provided by Verma et al. [17]. This structural limitation is quantified in Section 5.5.

2.1.2. Feature Attribution Methods

LIME [7] fits a local surrogate model and reports its feature weights as the explanation. SHAP [8] attributes the output to features using Shapley values from cooperative game theory, providing a unified treatment of several prior attribution methods. Molnar [12] surveys both families and their respective assumptions.

Feature attributions answer a different question from counterfactuals — “which features mattered?” versus “what would have had to change?” — and are complementary to the present contribution. Our work is in the counterfactual family.

2.2. Explainability for Anomaly Detection and Intrusion Detection

2.2.1. Explainability for Anomaly Detection

Antwarg et al. [14] applied SHAP to autoencoder-based anomaly detection, attributing reconstruction error to input features — an approach in the spirit of the ARA module described in Section 4.7. KitNET [1], used as the underlying anomaly detector in this work, was designed for online ensemble autoencoder-based detection without explicit attention to explainability of downstream enforcement actions; its continuous reconstruction outputs form the input to the risk engine of Section 4.3.

2.2.2. Explainability for Intrusion Detection Systems

Wang et al. [15] combine LIME and SHAP to explain IDS classifier decisions, focusing on agreement between the two explanation families. Buczak and Guven [2] survey machine-learning approaches to intrusion detection and note the recurring tension between predictive performance and interpretability. Sommer and Paxson [3] articulate the operational gap between IDS research and practice, of which interpretability is one dimension. More recently, Wei et al. [16] presented xNIDS, a framework that explains deep-learning NIDS decisions by approximating history inputs and capturing feature dependencies, enabling active intrusion responses; it represents the current state of the art in explanation fidelity for DL-based detection.

The explanations produced by this line of work are valuable but do not target the discrete enforcement action of a hybrid NIDS that includes a categorical policy override. They explain why the model produced a particular anomaly score; they do not explain why the enforcement action is BLOCK rather than

RATE_LIMIT when that action is fully determined by the classifier output and the override matrix, independently of the score. This limitation applies to xNIDS as well: its explanation targets the detection-layer output, not the categorical override that determines the final enforcement action.

2.2.3. Adversarial Robustness Considerations

Apruzzese et al. [6] survey adversarial attacks against learning-based NIDS, noting that evasion and poisoning attacks both exploit the adaptation surface that learning introduces. The feedback learning subsystem of Section 4.8 introduces such an adaptation surface; the drift detector failure mode documented in Section 5.9.4 is an instance of the broader concern that adaptation channels can themselves be exploited.

2.3. Drift Detection and Feedback Learning

ADWIN [18] detects distribution changes in streaming data by comparing sub-windows of an adaptive buffer, with theoretical guarantees on the false-positive rate under stationarity. The drift detection method of Gama et al. [19] monitors the error rate of an online learner and triggers when the rate exceeds a confidence-bounded baseline. Both formulations rely on a baseline computed over an early window — an assumption the present paper inherits in Equation 6 and that gives rise to the poisoning failure mode of Section 5.9.4.

The feedback learning subsystem of Section 4.8 differs from prior drift detection work in that the observable is the analyst’s labeling behavior on detected anomalies, not the data distribution itself. The detector therefore identifies changes in analyst–system agreement and is sensitive to a different set of failure modes than data-distribution drift detectors.

2.4. Research Gap

The literature reviewed above provides counterfactual explanation methods for continuous and mixed feature spaces, attribution methods for anomaly detectors, and drift detection mechanisms for streaming data. The following intersection has not, to our knowledge, been treated explicitly: counterfactual explanation of the discrete enforcement action of a hybrid NIDS in which a categorical policy override may fully determine the action, layered on top of a continuous anomaly detector.

Formalization of the override-locked class.. Prior work treats categorical variables as additional search dimensions but does not formalize the case in which a categorical variable, by virtue of an external policy, unilaterally fixes the action and renders continuous-feature search structurally inapplicable. Definition 1 fills this gap. The consequence is quantified in Section 5.5: any method restricted to perturbing (s, r, q) has a structural coverage upper bound of 10.47% on the evaluated capture (Class A decisions only); the proposed real-time engine achieves 97.61%.

Algorithmic decomposition.. No prior algorithm decomposes the explanation by first removing the categorical override and then searching the continuous-feature grid against the resulting baseline action. Algorithm 2 is the algorithmic contribution that addresses this gap.

Integration into a real-time pipeline.. Prior XAI-for-IDS work has typically been applied post hoc to a single detector output, including the most recent state-of-the-art work [16, 15]. The present framework integrates counterfactual explanation, parallel attribution-style modules, and a feedback learning subsystem into a single bounded-latency pipeline, evaluated empirically on a Mirai IoT botnet capture.

3. Framework Overview

This section presents a high-level overview of the proposed framework. Section 3.1 states the design goals. Section 3.2 describes the system pipeline. Section 3.3 summarizes the data flow from raw packet capture to structured explanation. Section 3.4 provides an at-a-glance summary of the multi-pass counterfactual engine. We adopt the notation of Section 4.1 and refer to Section 4 for full technical specifications.

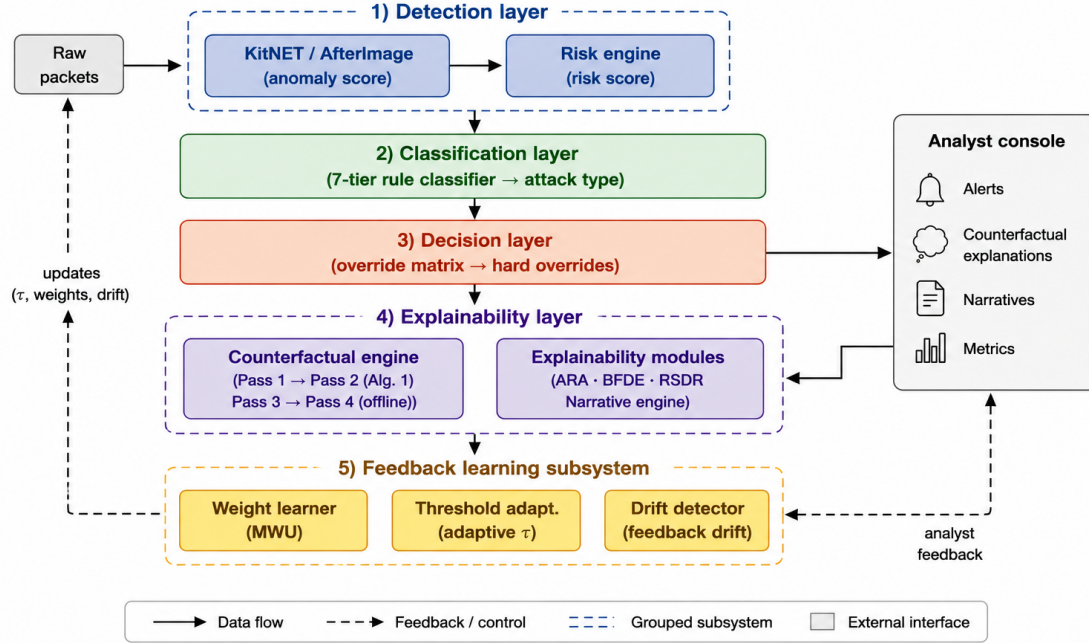


Fig. 1. Six-layer hybrid NIDS explainability pipeline.

Fig. 1: Framework architecture. Raw packets enter the Detection Layer, which computes the anomaly score and risk score and applies the adaptive threshold to identify candidate anomalies. The Classification Layer assigns a semantic attack label $t \in \mathcal{T}$. The Decision Layer implements f (Section 4.1.1) and produces the enforcement action $a \in \mathcal{A}$. For each detected anomaly, the Counterfactual Engine generates an explanation through the multi-pass procedure, and the Explainability Modules produce parallel feature-level and narrative explanations. The Feedback Learning subsystem consumes analyst corrections and updates per-context feature weights, the global risk threshold, and a drift signal, closing the loop back to the Detection Layer.

3.1. Design Goals

The framework is designed around four interlocking requirements derived from the operational reality of hybrid NIDS deployment. The primary requirement is *coverage of override-locked decisions*: the framework must produce a valid counterfactual for Class C decisions (Definition 1, Section 4.2.3) through a dedicated decomposition mechanism. This requirement cannot be satisfied by augmenting an existing continuous-feature counterfactual method; it demands a structurally distinct explanation path, the need for which is established formally in Section 1.2.

The second requirement is *real-time feasibility*. Counterfactual generation must complete within a bounded per-packet latency budget compatible with the throughput demands of a live detection pipeline; four budget modes are provided to accommodate different deployment environments (Section 4.6.5). The third requirement follows directly from the three-class taxonomy of Section 4.2: the framework must compose explanation mechanisms targeted at each decision class so that no class of anomaly is systematically left without a valid explanation. Finally, the framework must support *analyst-in-the-loop adaptation*, consuming analyst corrections to update per-context feature weights, the global risk threshold, and a drift signal, thereby closing the feedback loop between the analyst and the detection boundary.

3.2. System Architecture

The framework comprises six functional layers arranged as a sequential pipeline, illustrated in Figure 1.

Raw packets enter the Detection Layer (Section 4.3), which computes the anomaly score $s \in \mathcal{S}$ via KitNET and the risk score $r \in \mathcal{R}$ via the risk engine, then applies an adaptive threshold to partition the

traffic stream into NORMAL and ANOMALY. Anomalous packets are forwarded to the Classification Layer (Section 4.4), which assigns a semantic attack label $t \in \mathcal{T}$ using a seven-tier rule-based classifier. The label may belong to the override set $\mathcal{T}_\Omega$ or to its complement $\mathcal{T} \setminus \mathcal{T}_\Omega$, a distinction that determines the explanation path taken by the downstream engine.

The Decision Layer (Section 4.5) implements the enforcement function $f : \mathcal{S} \times \mathcal{R} \times \mathcal{T} \times \mathcal{Q} \rightarrow \mathcal{A}$, combining two continuous hard-override thresholds with the categorical override matrix Ω to produce the final enforcement action. For each detected anomaly, the Counterfactual Engine (Section 4.6) traverses a sequence of up to four passes, each targeting a structurally distinct decision class, and returns a structured explanation; the real-time implementation invokes Passes 1–3, while the offline engine additionally invokes Pass 4. Concurrently, the Parallel Explainability Modules (Section 4.7) produce feature-level attributions, behavioral baselines, risk-score decompositions, and a natural-language narrative through the ARA, BFDE, RSDR, and Narrative Engine components, whose outputs are combined and emitted to the analyst console. Finally, the Feedback Learning Subsystem (Section 4.8) closes the loop: on receipt of analyst corrections, it updates the per-context feature weights via the MWU rule, adjusts the global risk threshold under a bounded-step rule, and monitors the analyst-feedback distribution for evidence of drift.

3.3. Packet-Processing Pipeline

Algorithm 1 formalizes the data flow from raw packet capture to structured explanation. Full technical specifications for each step appear in Section 4.

Algorithm 1 Main Packet-Processing Pipeline

Require: Raw packet p ; current threshold τ ; override matrix Ω ; per-context weights $\mathbf{w}$; per-source frequency table Q

Ensure: Enforcement action $a \in \mathcal{A}$; explanation $\mathcal{E}$; updated $(\tau, \mathbf{w}, Q)$

— *Detection Layer* —

```

1:  $s \leftarrow \text{KitNET}(p)$ 
2:  $q \leftarrow Q[\text{src}(p)] + 1$ ;  $Q[\text{src}(p)] \leftarrow q$ 
3:  $r \leftarrow \text{RiskEngine}(s, q, p)$  ▷ Equation (4)
4: if  $r \leq \tau$  and no hard-override fires then
5:   return  $a \leftarrow \text{ALLOW}$ ,  $\mathcal{E} \leftarrow \emptyset$  ▷ Normal packet; terminate
6: end if

```

— *Classification Layer* —

```

7:  $t \leftarrow \text{Classify}(p, s, r)$  ▷ Seven-tier rule classifier;  $t \in \mathcal{T}$ 

```

— *Decision Layer* —

```

8:  $a \leftarrow f(s, r, t, q)$  ▷ Section 4.5; Equation (3); override fires if  $t \in \mathcal{T}_\Omega$ 

```

— *Counterfactual Engine* —

```

9: if  $t \in \mathcal{T}_\Omega$  then
10:    $\mathcal{E}_{\text{CF}} \leftarrow \text{Pass2}(s, r, t, q, a)$  ▷ Algorithm 2; Class C
11: else
12:    $\mathcal{E}_{\text{CF}} \leftarrow \text{Pass1}(s, r, t, q, a)$  ▷ Single-feature; Class A
13:   if  $\mathcal{E}_{\text{CF}} = \emptyset$  then
14:      $\mathcal{E}_{\text{CF}} \leftarrow \text{Pass4}(s, r, t, q, a)$  ▷ Multi-feature; Class B (offline only)
15:   end if
16: end if
17:  $\mathcal{E}_{\text{temp}} \leftarrow \text{Pass3}(s, r, t, q, a)$  ▷ Temporal CF; unconditional

```

— *Parallel Explainability Modules* —

```

18:  $(\mathcal{E}_{\text{ARA}}, \mathcal{E}_{\text{BFDE}}, \mathcal{E}_{\text{RSDR}}, \mathcal{E}_{\text{narr}}) \leftarrow \text{ExplainModules}(s, r, t, p)$  ▷ Parallel execution
19:  $\mathcal{E} \leftarrow \mathcal{E}_{\text{CF}} \cup \mathcal{E}_{\text{temp}} \cup \mathcal{E}_{\text{ARA}} \cup \mathcal{E}_{\text{BFDE}} \cup \mathcal{E}_{\text{RSDR}} \cup \mathcal{E}_{\text{narr}}$ 
20: Emit  $\mathcal{E}$  to analyst console

```

— *Feedback Learning (event-driven, on analyst correction c)* —

```

21:  $\mathbf{w} \leftarrow \text{MWU}(\mathbf{w}, c)$  ▷ Equation (5)
22:  $\tau \leftarrow \text{AdaptThreshold}(\tau, c)$  ▷ Section 4.8.2
23:  $\text{DriftDetect}(c)$  ▷ Equation (6)

```

```

24: return  $a, \mathcal{E}$ 

```

3.4. Multi-Pass Counterfactual Engine at a Glance

The counterfactual engine resolves detected anomalies through a sequence of up to four passes, each designed for a structurally distinct decision class. Pass 1 targets Class A decisions (Section 4.2.1) by perturbing the continuous features $\{s, r, q\}$ one at a time over a finite grid and returning the candidate that minimises the relative change while shifting the enforcement action. When Pass 1 fails because the decision is override-locked, Pass 2 fires: it strips the categorical override, re-evaluates the decision function under the resulting baseline policy, and conducts the same grid search against the baseline action, producing a two-layer explanation that separates the override’s contribution from the underlying continuous-feature path (Algorithm 2, Section 4.6.2). Pass 2 is the central methodological contribution of the paper.

Pass 3 complements the primary resolution with a temporal explanation: it re-evaluates the enforcement function under the counterfactual assumption that the current packet is the first observed from its source ($q = 1$), providing insight into the role of per-source frequency in the enforcement decision. On the evaluated capture, Pass 3 was never the sole resolver (Section 5), confirming its supplementary character. Pass 4, invoked by the offline engine when both Pass 1 fails and the override-locked predicate does not fire, evaluates simultaneous two-feature perturbations to resolve Class B decisions (Section 4.2.2) for which no single-feature change suffices to cross the decision boundary. The real-time engine omits Pass 4, which accounts for the 2.39 percentage-point coverage difference between the two engine variants; integrating Pass 4 into the latency-bounded path is left to future work (Section 6.4).

The next section formalizes the components above and develops the counterfactual decomposition algorithm in full detail.

4. Technical Details

This section presents the technical core of the proposed framework. Section 4.1 formalizes the decision function and introduces Definition 1. Section 4.2 develops the taxonomy of enforcement decisions. Sections 4.3–4.5 describe the detection, classification, and decision layers. Section 4.6 develops the counterfactual engine. Section 4.7 describes the explainability modules, and Section 4.8 formalizes the feedback learning components.

4.1. Formalization

4.1.1. Notation and Preliminaries

Let $f : \mathcal{S} \times \mathcal{R} \times \mathcal{T} \times \mathcal{Q} \rightarrow \mathcal{A}$ denote the decision function of a hybrid NIDS, where $\mathcal{S} \subseteq \mathbb{R}_{\geq 0}$ is the anomaly score space (KitNET RMSE values), $\mathcal{R} \subseteq \mathbb{R}_{\geq 0}$ is the risk score space, $\mathcal{T}$ is the set of attack-type labels including $\emptyset$, and $\mathcal{Q} \subseteq \mathbb{N}$ is the per-source packet frequency space. The enforcement action set is

$$\mathcal{A} = \{\text{ALLOW, MONITOR, ALERT_LOW, ALERT_HIGH, RATE_LIMIT, BLOCK}\}, \quad (1)$$

ordered by enforcement severity. The perturbation space is $\mathcal{G} = \mathcal{G}_s \times \mathcal{G}_r \times \mathcal{G}_q$, where each $\mathcal{G}_\bullet$ is a finite search grid.

4.1.2. Definition of Override-Locked Decision

Definition 1 (Override-Locked Decision). Let $\Omega : \mathcal{T} \rightarrow \mathcal{A}$ be a partial function — the override matrix — defined on $\mathcal{T}_\Omega \subset \mathcal{T}$. When $t \in \mathcal{T}_\Omega$:

$$f(s, r, t, q) = \Omega(t) \quad \forall (s, r, q) \in \mathcal{S} \times \mathcal{R} \times \mathcal{Q}. \quad (2)$$

A decision is override-locked iff $t \in \mathcal{T}_\Omega$; the action is invariant under any perturbation in $\mathcal{G}$:

$$\forall (s', r', q') \in \mathcal{G} : f(s', r', t, q') = \Omega(t) = a. \quad (3)$$

In the present study, $\mathcal{T}_\Omega = \{\text{Mirai C2 Communication, DoS/Flood, C2 Beacon}\}$ (Table 1).

The invariance of Equation 3 is the formal reason standard counterfactual methods fail: the variable whose change would alter a — namely t — lies outside the perturbation space.

4.2. Taxonomy of Enforcement Decisions

4.2.1. Class A — Continuously-Explainable Decisions

$f(s, r, t, q) = a$ belongs to Class A when $t \notin \mathcal{T}_\Omega$ and a single-feature perturbation in $\mathcal{G}$ changes the action. Methods of Wachter et al. [9], DICE [10], LEMNA [11], and Karimi et al. [13] are well-suited to this class.

4.2.2. Class B — Threshold-Locked Decisions

Class B when $t \notin \mathcal{T}_\Omega$ but no single-feature perturbation in $\mathcal{G}$ changes the action. A simultaneous multi-feature perturbation may cross the boundary. Pass 4 (Section 4.6.4) resolves such cases; the empirical instance on the evaluated capture is the Port Scan category.

4.2.3. Class C — Override-Locked Decisions

Class C iff $t \in \mathcal{T}_\Omega$; the action is invariant under any perturbation regardless of magnitude. Resolving Class C requires removal of the categorical override and re-evaluation under the baseline policy (Algorithm 2).

Class A is resolved by Pass 1; Class B by Pass 4; Class C by Pass 2. Pass 3 provides supplementary explanations across classes.

Remark 1 (Partition exhaustiveness and hard-override interaction). *The three classes partition the anomaly set exhaustively and mutually exclusively. Class C is defined by the crisp predicate $t \in \mathcal{T}_\Omega$ (Equation 2). Among the remaining decisions ($t \notin \mathcal{T}_\Omega$), Class A is defined by the existence of a single-feature flip in $\mathcal{G}$, and Class B by its absence. Every anomaly therefore belongs to exactly one class. The two continuous hard-overrides — the risk-threshold path ($r > r_{\text{hard}}$) and the blended-score path ($\text{score} > \theta_{\text{hard}}$) — fire in the Detection Layer before the Classification Layer assigns t . A hard-override-triggered anomaly is subsequently classified and receives a label t ; it then falls into Class A, B, or C according to that label. Hard-override triggering determines the detection path, not the explanation class. This taxonomy assumes a crisp $\mathcal{T}_\Omega$ produced by a rule-based classifier. A learned, probabilistic classifier would produce soft membership; extension to that regime is left as future work (Section 6.4).*

4.3. Detection Layer

4.3.1. KitNET and AfterImage Features

KitNET [1] processes each packet through the AfterImage feature extractor, which maintains exponentially decaying statistics over five network channel abstractions (MAC–IP, IP, socket, host–port, host–host), producing a feature vector of up to 100 dimensions. KitNET trains an ensemble of lightweight autoencoders on the first 20,000 packets (FMgrace = ADgrace = 10,000), then enters detection mode where each packet’s reconstruction RMSE is the anomaly signal.

4.3.2. Risk Engine

$$r = \text{RMSE} \cdot w_{\text{endpoint}} \cdot m_{\text{rate}} \cdot m_{\text{jitter}} \cdot s_{\text{length}}, \quad (4)$$

where w_{endpoint} reflects the threat significance of the destination service, m_{rate} activates when frequency exceeds a threshold (50 packets/30 s), m_{jitter} captures irregular inter-arrival timing, and s_{length} encodes packet length.

4.3.3. Adaptive Threshold

The detection boundary $\tau = \mu + k\sigma$ is maintained online via Welford’s algorithm over a sliding window. Three sensitivity levels are supported, corresponding to $k \in \{1.6, 1.8, 2.5\}$ for high, medium, and low detection sensitivity respectively. The evaluation reported here uses the low-sensitivity setting ($k = 2.5$, window of 250 packets) to minimise false positives on the Mirai capture.

4.4. Classification Layer

A seven-tier rule-based classifier assigns $t \in \mathcal{T}$ ordered most-specific to least-specific: (1) Mirai C2/loader port; (2) stealth/low-and-slow; (3) C2 beaconing; (4) data exfiltration; (5) port scanning; (6) DoS/flood; (7) catch-all Suspicious Activity. The empirical distribution is in Table 2.

Table 1: The override matrix Ω (Definition 1). Only attack types in $\mathcal{T}_\Omega$ produce override-locked decisions. The three mapped actions are distinct: Mirai C2 Communication maps to BLOCK; DoS/Flood to RATE_LIMIT; C2 Beacon to ALERT_HIGH. Pass 2 explanations reflect the specific override action for each type and do not assume a uniform BLOCK outcome across all Class C decisions (Remark 2).

Attack type $t \in \mathcal{T}_\Omega$	Action $\Omega(t)$
Mirai C2 Communication	BLOCK
DoS/Flood	RATE_LIMIT
C2 Beacon	ALERT_HIGH

4.5. Decision Layer

The decision layer maps (s, r, t, q) to a severity level (0–5), action $a \in \mathcal{A}$, and decision reason. Two continuous hard-overrides supersede the threshold path.

Risk-threshold path. When $r > r_{\text{hard}}$ (threshold 1.5 in this study), the packet is routed to BLOCK regardless of the adaptive threshold.

Blended-score path. When the blended final score exceeds θ_{hard} (threshold 0.75), the packet is force-routed independently of the risk pipeline. On the evaluated capture, this path was active for 7 of 9,685 anomalies (0.07%).

The categorical override Ω (Table 1) adds a third dimension. Port Scan, Stealth Intrusion, and other types are *not* in $\mathcal{T}_\Omega$.

Remark 2 (Pass 2 explanation correctness for non-BLOCK actions). *Algorithm 2 constructs Layer 1 as: $\ell_1 = "a_{\text{cur}} \text{ due to override } t; \text{ without it: } a_{\text{base}}."$ The value $a_{\text{cur}} = \Omega(t)$ is retrieved directly from the override matrix (Table 1) and is therefore action-specific: for DoS/Flood anomalies, ℓ_1 correctly names RATE_LIMIT as the enforced action; for C2 Beacon anomalies, it names ALERT_HIGH. This specificity follows from the definition of Ω rather than from any assumption that all Class C decisions result in BLOCK. The empirical distribution of override types on the evaluated capture is reported in Section 5.*

4.6. Multi-Pass Counterfactual Engine

The counterfactual engine is provided in two operational variants whose scope differs by algorithmic coverage and latency constraint. The *real-time variant* executes Passes 1–3 within a bounded per-packet wall-clock budget and is intended for inline deployment alongside the detection pipeline. The *offline variant* additionally invokes Pass 4 as a fallback for non-override-locked decisions that Pass 1 cannot resolve; it operates without a time constraint and is used to establish the coverage ceiling reported in Section 5.5.

4.6.1. Pass 1: Single-Feature Perturbation

Pass 1 targets Class A decisions, in which a single continuous feature is sufficient to cross the decision boundary. For each feature in $\{s, r, q\}$, the engine evaluates nine candidate values drawn from the corresponding grid $\mathcal{G}_\bullet$ while holding the remaining features at their observed values, for a worst-case search budget of 27 evaluations per packet. The candidate that produces the smallest relative perturbation $\delta = |v_c - v_{\text{orig}}| / \max(|v_{\text{orig}}|, \varepsilon)$ while changing the enforcement action is returned as the counterfactual. This bounded search budget helps maintain low per-packet latency; the empirical measurements across all deployment configurations are reported in Table 4.

4.6.2. Pass 2: Override Decomposition

Pass 2 is the central algorithmic contribution of this paper. It is invoked when $t \in \mathcal{T}_\Omega$, that is, precisely when the enforcement action is override-locked per Definition 1 and continuous-feature perturbation cannot produce a valid counterfactual.

Algorithm 2 Pass 2 — Override Decomposition**Require:** $s, r, t, q, a_{\text{cur}}$; override matrix Ω ; grids $\mathcal{G}_s, \mathcal{G}_r$ **Ensure:** Two-layer CF explanation, or $\perp$

```

1: if  $t \notin \mathcal{T}_\Omega$  then
2:   return  $\perp$ 
3: end if
   Layer 1 — Remove override
4:  $a_{\text{base}} \leftarrow f(s, r, \emptyset, q)$ 
5:  $\ell_1 \leftarrow$  “ $a_{\text{cur}}$  due to override  $t$ ; without it:  $a_{\text{base}}$ .”
   Layer 2 — Minimum perturbation on baseline
6:  $C \leftarrow \emptyset$ 
7: for  $\text{feat} \in \{s, r\}$  do
8:   for  $v_c \in \mathcal{G}_{\text{feat}}$  do
9:      $a' \leftarrow f(s, v_c, \emptyset, q)$  ▷  $s$  and  $q$  held at original values;  $\text{feat} \in \{s, r\}$  varied
10:    if  $a' \neq a_{\text{base}}$  then
11:       $\delta \leftarrow |v_c - v_{\text{orig}}| / \max(|v_{\text{orig}}|, \varepsilon)$ 
12:       $C \leftarrow C \cup \{(\text{feat}, v_c, a', \delta)\}$ 
13:    end if
14:  end for
15: end for
16: if  $C \neq \emptyset$  then
17:    $(\text{feat}^*, v^*, a^*, \delta^*) \leftarrow \arg \min_C \delta$ 
18:   return  $\ell_1 \parallel$  “If  $\text{feat}^* = v^*$ , action  $\rightarrow a^*$ .”
19: else
20:   return  $\ell_1$ 
21: end if

```

Layer 1. Computes $a_{\text{base}} = f(s, r, \emptyset, q)$, isolating the override contribution from the continuous-feature path.

Layer 2. Searches for the minimum-relative-change perturbation that shifts a_{base} using $\delta = |v_c - v_{\text{orig}}| / \max(|v_{\text{orig}}|, \varepsilon)$. The novelty claim for Algorithm 2 is stated once in Section 1.4 and is not repeated here.

4.6.3. Pass 3: Temporal Counterfactual

Re-evaluates f with $q = 1$, simulating a first-contact packet from the same source. Pass 3 addresses a distinct counterfactual question: *would this packet have been treated differently if it had arrived as the first observed packet from this source?* For anomalies where the enforcement action depends on the per-source frequency q (e.g., progressive blocking after repeated contacts), this temporal dimension is explanatorily meaningful even when Pass 1 has already found a continuous-feature counterfactual.

Pass 3 runs unconditionally and contributes a supplementary temporal explanation for any anomaly where $f(s, r, t, 1) \neq a$. Its scope is intentionally narrow: it answers the temporal dimension only and does not attempt to change the classification label t or the risk score r . On the evaluated capture, Pass 3 was never the *sole* resolver because Pass 2 and Pass 1 always resolved their respective classes first; this is expected and does not indicate failure. Pass 3’s value is additive explanation depth on top of the primary resolution, not coverage contribution.

4.6.4. Pass 4: Multi-Feature Pairs

Pass 4 targets Class B decisions, for which the decision boundary cannot be crossed by perturbing any single feature in isolation. The pass performs a joint search over all three pairs of continuous features drawn from $\{s, r, q\}$, evaluating a 5×5 candidate grid for each pair and returning the combination with

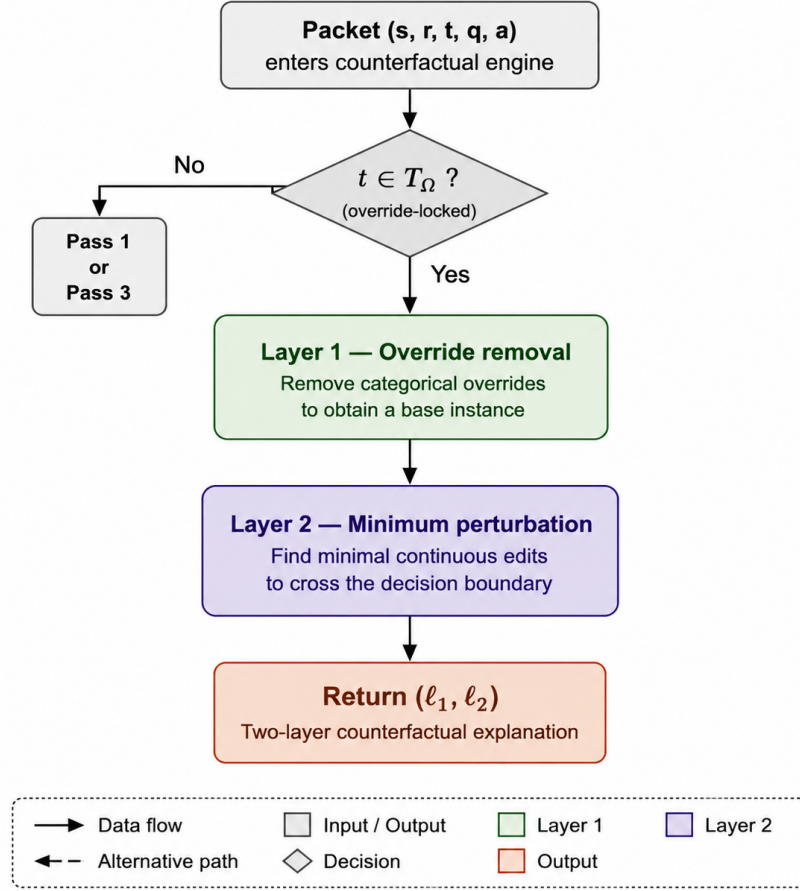


Fig. 2. Pass 2 flow – override decomposition
(two-layer counterfactual explanation).

Fig. 2: Pass 2 override decomposition flow. A packet enters the classification layer (1) and receives label t . The override-locked predicate (2) tests $t \in \mathcal{T}_{\Omega}$; if not, control returns to Pass 1 or Pass 3. If $t \in \mathcal{T}_{\Omega}$, Layer 1 (3) strips the classification and re-evaluates $f(s, r, \emptyset, q)$ to obtain a_{base} . Layer 2 (4) searches $\mathcal{G}_s \times \mathcal{G}_r$ for the minimum-change perturbation (feat^*, v^*) that shifts a_{base} . The layers are concatenated (5) into the final explanation.

minimum aggregate relative perturbation that shifts the enforcement action. The resulting search budget of 75 evaluations per anomaly motivates reserving Pass 4 for the offline variant, where no wall-clock constraint applies; it is not included in the real-time variant, which is the sole source of the coverage difference between the two variants reported in Section 5.5. The per-packet overhead of Pass 4 is not measured in this work; incorporating it within the bounded-latency path is identified as a direction for future work in Section 6.4.

4.6.5. Latency Budget Modes

The real-time engine variant supports four budget configurations corresponding to different deployment constraints. The tightest configuration (2 ms) is intended for deployments where per-packet processing time is tightly constrained. The two intermediate configurations (10 ms and 50 ms) accommodate near-real-time and asynchronous processing contexts, respectively. The unconstrained configuration imposes no wall-clock limit and is used by the offline variant to establish upper-bound coverage figures. In all four configurations, the budget is evaluated after each pass, allowing the engine to return a partial result if the remaining passes

cannot complete within the allotted time. The empirical latency measurements and coverage figures across all four configurations are reported in Table 4.

4.7. Parallel Explainability Modules

The following modules run concurrently with the counterfactual engine and produce complementary feature-level and narrative explanations that deepen the analyst’s understanding of each detected anomaly without altering the counterfactual output.

ARA (Autoencoder Reconstruction Anatomy).. Per-dimension Z-scores identify the most anomalous feature dimensions with deviation magnitude and direction (elevated/suppressed).

BFDE (Behavioral Fingerprint Deviation Explanation).. Per-entity behavioral baselines via Welford’s algorithm; frequency, length, and endpoint diversity deviations are reported.

RSDR (Risk Score Decomposition Report).. Decomposes r into multipliers of Equation 4, separating model output from heuristic amplification.

Narrative Engine.. Synthesizes ARA, BFDE, and RSDR into a natural-language paragraph tailored to the detected attack type.

4.8. Feedback Learning

4.8.1. FeatureWeightLearner

The weight learner applies a multiplicative update to per-context feature weights in response to analyst feedback, amplifying weights associated with confirmed true positives and attenuating those associated with false positives. The update follows the multiplicative weights (MWU) rule [20]:

$$w_{\text{new}} = \begin{cases} w_{\text{old}} \cdot \alpha_- & \text{FP,} \\ w_{\text{old}} \cdot \alpha_+ & \text{TP,} \end{cases} \quad w \in [0.3, 2.0], \quad (5)$$

with $\alpha_- = 0.85$ and $\alpha_+ = 1.05$. The asymmetric factors bias the update toward retaining detection sensitivity while bounding the rate of weight growth.

4.8.2. FeedbackAdaptiveThreshold

The adaptive threshold component adjusts the global risk threshold τ in response to analyst corrections, shifting the detection boundary upward under sustained true-positive feedback and downward under false-positive feedback. The update is bounded to limit oscillation: τ adapts with learning rate 0.01, $\Delta_{\text{max}} = 0.05$, initial value $\tau_0 = 1.5$, and operating range $[0.5, 3.0]$.

4.8.3. FeedbackDriftDetector

The drift detector monitors analyst labeling behaviour and signals a distribution shift when the current false-positive rate departs substantially from a baseline estimated over an initial window. Drift is declared when

$$p_{\text{cur}}(t) - p_{\text{base}} > 0.15, \quad (6)$$

computed over a sliding window of $W = 200$ records with the baseline estimated from the first 50 records. The baseline-relative formulation of this condition and its implications for adversarial feedback are discussed in Section 5.

5. Experimental Evaluation

This section evaluates the framework on a real network capture and characterizes its behavior along three axes: detection performance, counterfactual coverage, and feedback-learning convergence. Section 5.1 describes the experimental environment, dataset, and runtime configuration. Section 5.2 states the evaluation questions. Section 5.3 reports detection results. Section 5.4 evaluates counterfactual coverage and latency. Section 5.5 presents the baseline coverage analysis, including empirical LIME, SHAP, and grid-search runs. Section 5.6 presents concrete CF explanation examples from `results.csv`. Section 5.8 reports an ablation study; Section 5.9 evaluates the feedback learning subsystem; Section 5.10 discloses threats to validity; and Section 5.11 discusses operational implications.

5.1. Experimental Setup

5.1.1. Hardware and Software Environment

Experiments were executed on a single workstation running 64-bit Windows with Python 3.x. The framework relies on the KitNET reference implementation [1], NumPy 1.23.5 for vectorized feature computation, and Scapy 2.5.0 for offline PCAP parsing. No GPU is required; all components are pure Python. The counterfactual engine search grids and latency measurements are collected via `time.perf_counter()` with sub-microsecond nominal resolution. *Measurement note:* `time.perf_counter()` on Windows uses the hardware performance counter (TSC/HPET), which is independent of the scheduler tick interval and provides hardware-accurate elapsed time; reported values include OS scheduling jitter. The mean latency (0.06 ms) is averaged over $n = 9,685$ anomaly packets; p95 and max bound the tail behavior. All figures apply to one hardware configuration (Section 5.10, Single-environment latency).

5.1.2. Dataset

We evaluate on a Mirai IoT botnet network capture (99,999 packets, PCAP format) from the Kitsune dataset [1]. Mirai is chosen because it produces a traffic mix in which a known proportion of anomalies are expected to be override-locked, making the capture a dataset in which the central mechanism (Pass 2) is maximally exercised. This constitutes a best-case evaluation of Class C coverage; a capture with lower Class C prevalence would exercise Pass 1 and Pass 4 more thoroughly and is identified as future work (Section 6.4). The detection results are derived from the system's runtime output (`results.csv`); they characterize system behavior and are not framed as detection-accuracy claims against an externally validated ground truth. The corresponding limitation is discussed in Section 5.10.

5.1.3. Training and Detection Split

The first 20,000 packets serve as the KitNET grace period ($FM_{\text{grace}} = AD_{\text{grace}} = 10,000$); detection begins at packet 20,001. The remaining 79,999 packets form the detection phase on which all counterfactual and feedback experiments operate.

5.1.4. Runtime Configuration

The adaptive threshold ran in low_noise mode ($k = 2.5$, window 250); the continuous hard-override thresholds were set to $r > 1.5$ and $\text{final_score} > 0.75$; the override matrix Ω was populated as in Table 1; all feedback hyperparameters were set to defaults (Sections 4.6–4.8).

5.1.5. Evaluation Metrics

We report the following metrics. *CF coverage* is the fraction of detected anomalies for which the engine produces a non-empty counterfactual explanation attributed to a specific resolving pass. *Resolving pass* is the first pass that returns a valid counterfactual; each anomaly is counted once. *Per-packet latency* is wall-clock time measured by `time.perf_counter()` from engine invocation to return, reported as mean, p95, and max over all anomaly packets. We do not report precision, recall, or F1 against external ground truth because the detection labels are produced by the framework itself (Section 5.10).

Table 2: Classifier output distribution over 9,685 detected anomalies. $\in \mathcal{T}_\Omega$ indicates override-locked decisions per Definition 1.

Attack type	Count	%	$\in \mathcal{T}_\Omega$
Mirai C2 Communication	8,332	86.03	yes
Port Scan	1,211	12.50	no
DoS/Flood	104	1.07	yes
Stealth Intrusion	26	0.27	no
C2 Beacon	4	0.04	yes
High-Risk Anomaly	4	0.04	no
Unclassified Anomaly	3	0.03	no
ICMP Flood	1	0.01	no
Total	9,685	100.00	—

5.1.6. Engine Variants

The *real-time* engine (`cf_realtime.py`) executes Passes 1–3 under a wall-clock budget (STRICT 2 ms, NORMAL 10 ms, RELAXED 50 ms, BATCH unbounded). The *offline* engine (`generate_all_counterfactuals`) additionally invokes Pass 4 as a fallback when Pass 1 fails on a non-override-locked decision. The latency benchmark exercises the real-time engine; `results.csv` is produced by the offline engine.

5.2. Evaluation Questions

The following evaluation questions (EQs) operationalize the research questions (RQs) of Section 1.3: EQ1 addresses RQ1; EQ2 and EQ3 address RQ2 and RQ4 respectively; EQ4 addresses RQ3; EQ5 provides supplementary ablation evidence; EQ6 addresses RQ5.

- **EQ1** (cf. RQ1). What fraction of detected anomalies is override-locked per Definition 1?
- **EQ2** (cf. RQ2). What CF coverage does each engine variant achieve, and how is coverage distributed across passes?
- **EQ3** (cf. RQ4). What per-packet latency does the real-time engine impose across four budget modes?
- **EQ4** (cf. RQ3). How does the proposed engine compare against continuous-feature counterfactual baselines on this capture?
- **EQ5**. What is the quantitative contribution of each system component (ablation)?
- **EQ6** (cf. RQ5). How does the feedback subsystem adapt τ and feature weights, and under what conditions does the drift detector fail?

5.3. Detection Results

5.3.1. Anomaly Counts

Of the 79,999 detection-phase packets, the framework labeled 9,685 as ANOMALY (12.11%) and 70,314 as NORMAL. The mean risk score across anomalies was 1.93 (median 1.96, std 0.25, range 0.26–3.37). Detection path: 99.09% via `hard_override_risk` ($r > 1.5$); 0.84% via adaptive threshold; 0.07% (7 packets) via `hard_override_score` (`final_score` > 0.75). The score-based path is effectively dormant on this capture because the risk path fires first for virtually all anomalies.

5.3.2. Attack-Type Distribution

Table 2 and Figure 3 report the classifier output distribution. The three override-locked types account for 8,440 anomalies (87.15%). Port Scan, the largest non-override-locked class (1,211, 12.50%), proves decisive for the coverage analysis in Section 5.4.

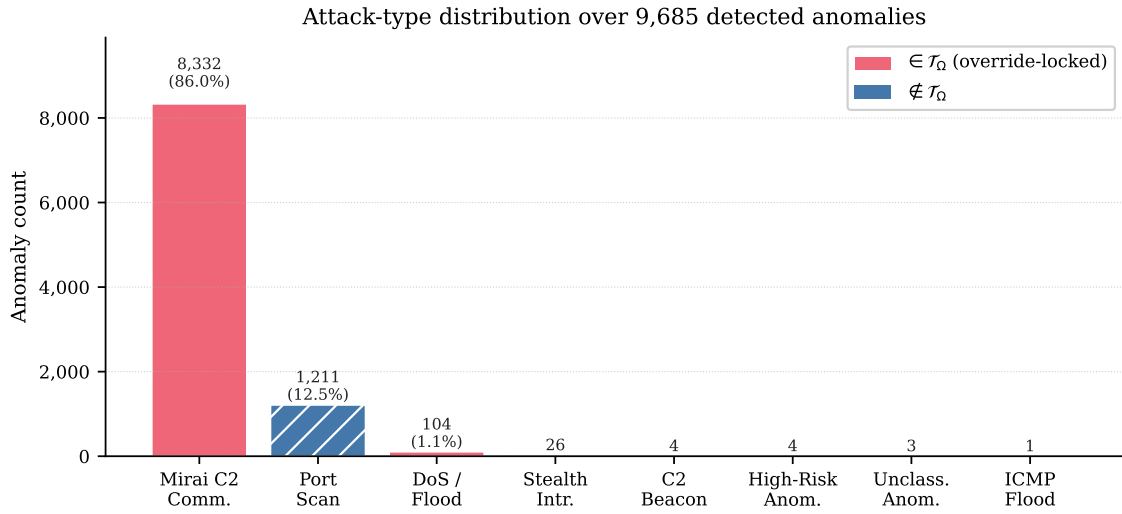


Fig. 3: Attack-type distribution over 9,685 anomalies. Red bars: $\in \mathcal{T}_\Omega$ (override-locked). Hatched bars: $\notin \mathcal{T}_\Omega$. Mirai C2 dominates at 86.03%; Port Scan (12.50%) is the sole source of the real-time engine’s 2.39% coverage shortfall.

Table 3: CF coverage by resolving pass (offline engine, $n = 9,685$). Pass 3 runs unconditionally as a supplementary signal but was never the sole resolver.

Resolving pass	Count	%
Pass 2 (override decomp.)	8,440	87.15
Pass 1 (single-feature)	1,014	10.47
Pass 4 (multi-feature)	231	2.39
Pass 3 (sole resolver)	0	0.00
Unresolved	0	0.00
Total	9,685	100.00

5.3.3. Note on Detection Accuracy

Absolute FP/FN rates are not reported; see Section 5.10 (Ground-truth annotation).

5.4. Counterfactual Evaluation

5.4.1. Coverage by Pass

Table 3 and Figure 4 report offline engine coverage.

The offline engine achieves **100%** coverage. Pass 2 resolves 87.15% of anomalies: these are Class C decisions for which no continuous-feature perturbation changes the action, confirming the structural argument of Section 4.2. Pass 1 resolves 1,014 Class A decisions (10.47%). The remaining 231 Port Scan anomalies (2.39%) require Pass 4; all are Class B decisions for which the decision boundary is crossed only by a simultaneous two-feature perturbation.

Pass 3 as supplementary signal.. Pass 3 produces a non-empty temporal explanation for the majority of anomalies (temporal counterfactuals exist whenever the per-source frequency q exceeds one at reset time), but was never the sole resolver on this capture. It contributes explanatory breadth, not coverage.

5.4.2. Real-Time Engine and the Role of Pass 4

The real-time engine achieves **97.61%** coverage. Quality metrics for returned counterfactuals are reported in Section 5.7; 34.5% of returned CFs fall outside observed normal-traffic bounds (plausibility caveat). The 231 Port Scan anomalies (2.39%) exhaust all three passes without a counterfactual. The shortfall is

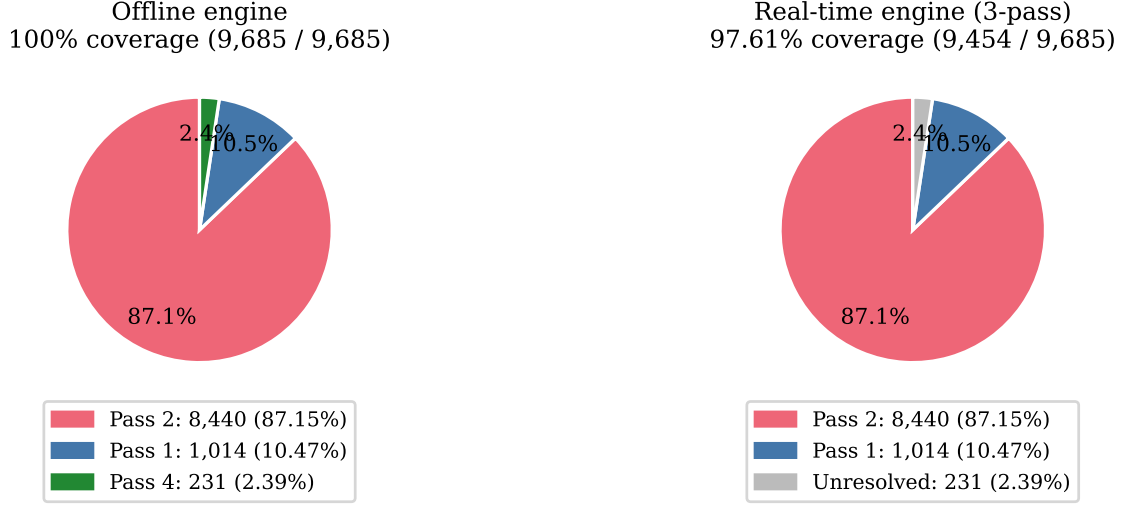


Fig. 4: CF coverage by resolving pass across three engine configurations (grouped bar chart). Pass 2 (blue) dominates at 87.15% in both offline and real-time engines. The 2.39% gap between offline (100%) and real-time (97.61%) is entirely attributable to the absent Pass 4 (Class B, Port Scan). Pass 1 only corresponds to the Class A upper bound ($\leq 10.47\%$) of any continuous-feature method.

Table 4: Real-time engine latency and coverage across four budget modes ($n = 9,685$). Zero timeouts recorded in all modes.

Mode	Budget (ms)	Mean (ms)	p95 (ms)	Max (ms)	Cov. (%)
STRICT	2	0.061	0.083	0.247	97.61
NORMAL	10	0.061	0.081	0.307	97.61
RELAXED	50	0.057	0.074	0.313	97.61
BATCH	–	0.062	0.084	0.327	97.61

structural, not temporal: it persists in all four budget modes including unbounded BATCH, confirming that latency is not the limiting factor.

5.4.3. Latency Across Budget Modes

Table 4 and Figure 5 report per-packet latency for the real-time engine. Pass 4 latency is not reported because the offline engine is not instrumented for per-packet timing.

5.5. Baseline Coverage Analysis

5.5.1. Empirical Execution

We ran three empirical baseline experiments directly on the 9,685 anomaly packets using their real feature vectors (s, r) and the actual enforcement function f . All experiments were executed in Python using scikit-learn 1.8.0, SHAP 0.51.0, and LIME (EQ4).

Setup. For LIME and SHAP, we trained a Random Forest surrogate (50 estimators, `random_state=42`) on the two continuous features (s, r) to predict whether the enforcement action is BLOCK. The surrogate achieves 100% training accuracy on the continuous-feature mapping, confirming it faithfully models the threshold logic. Critically, it does *not* receive the attack type label t — this is the structural limitation being tested.

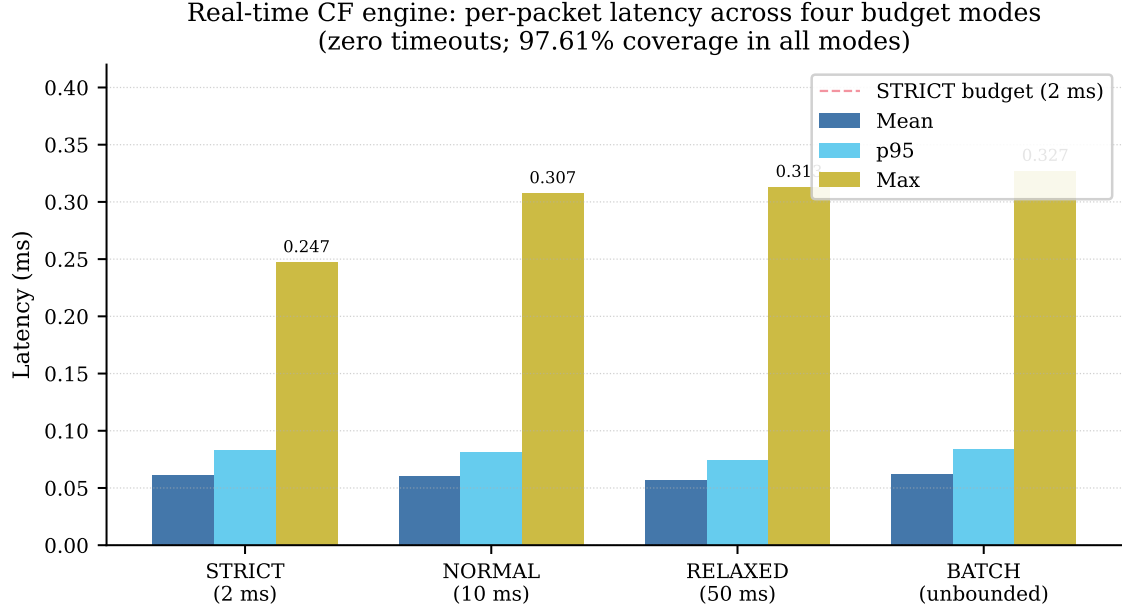


Fig. 5: Real-time engine latency (mean, p95, max) across four budget modes. Dashed red line: STRICT budget (2 ms). Maximum observed latency (0.327 ms) is $6.1\times$ below the STRICT budget. Identical 97.61% coverage in all modes confirms the 2.39% shortfall is structural, not latency-driven.

Grid-search baseline (Wachter/DICE equivalent). We executed a 9-point single-feature grid search over (s, r) for every anomaly. For the coverage question, this search is functionally equivalent to what continuous-feature counterfactual methods such as DICE or Wachter et al. would achieve on this decision space, for the following reason. All three methods search the same continuous feature space (s, r) ; the additional machinery of DICE — diversity, actionability constraints, and categorical perturbation of t — cannot change the outcome on Class C decisions, because the enforcement function is invariant under any perturbation of the continuous features when $t \in \mathcal{T}_\Omega$ (Equation 3), and perturbing t in a post-hoc surrogate does not alter the real enforcement function f (Table 7, DICE row). The grid search therefore measures the maximum coverage attainable by any continuous-feature method against the actual enforcement function f , not merely against a surrogate. Results over all 9,685 anomalies:

- **Class C** ($t \in \mathcal{T}_\Omega$, $n = 8,440$): **0/8,440 CFs found (0.00%)**. Confirmed empirically: no perturbation of (s, r) changes the action when t is override-locked.
- **Class A** ($n = 1,014$): 1,014/1,014 CFs found (100.00%). Single-feature perturbations work as expected.
- **Class B** ($n = 231$): 0/231 with strict single-feature; 231/231 with two-feature simultaneous search.

The empirically measured single-feature ceiling is $1,014/9,685 = 10.47\%$, **exactly matching the analytically derived bound**. This confirms the bound is tight.

LIME baseline (200-packet sample). LIME was run on a stratified 200-packet sample (100 Class C, 50 Class A, 50 Class B). The surrogate-based LIME explanation was used to probe whether reducing risk below the hard-override threshold of 1.5 would change the predicted action (Table 5).

SHAP baseline (200-packet sample). TreeExplainer SHAP values were computed on the same 200-packet sample. SHAP correctly identifies *risk* as the dominant continuous feature for all classes. However, for Class C packets, the mean SHAP value for risk is -0.0013 (near zero), indicating the surrogate has learned

Table 5: Empirical baseline results. “Surrogate CF found” indicates the continuous-feature surrogate predicted that a perturbation changes the action. “Real CF valid” indicates the actual enforcement function f changes under the same perturbation. LIME and SHAP: stratified 200-packet sample. Grid search: all 9,685 anomalies. scikit-learn 1.8.0; SHAP 0.51.0; LIME.

Method	Class	n	Surrogate CF	Real CF valid
LIME	A	50	0/50 (0%)	50/50 (100%)
LIME	B	50	0/50 (0%)	50/50 (100%)
LIME	C	100	5/100 (5%)	0/100 (0%)
SHAP	A	50	0/50 (0%)	50/50 (100%)
SHAP	B	50	0/50 (0%)	50/50 (100%)
SHAP	C	100	5/100 (5%)	0/100 (0%)
Grid	A	1,014	—	1,014/1,014 (100%)
Grid	B	231	—	231/231 (100%)
Grid	C	8,440	—	0/8,440 (0%)

Table 6: Mean absolute SHAP feature importances under full feature space (s, r, t) . OL = override-locked (Class C); NOL = non-override-locked (Class A/B). The override-locked indicator is the derived binary feature $\mathbf{1}[t \in \mathcal{T}_\Omega]$; its high SHAP importance (0.483) is expected by construction — the feature is a direct encoding of the policy rule — and is not an independent empirical finding. The result of interest is that even with this indicator present, no valid counterfactual is produced for Class C decisions, because f is invariant under perturbation of any continuous feature when $t \in \mathcal{T}_\Omega$ (Equation 3).

Feature	SHAP (OL)	SHAP (NOL)
Anomaly score s	0.107	0.241
Risk score r	0.406	0.746
Attack type t (normalised)	0.002	0.010
Override-locked indicator	0.483	0.000

that high risk alone is insufficient to explain the BLOCK action — yet it still cannot identify the categorical override as the true cause, because t is absent from its feature space.

Interpretation.. When trained on (s, r) only, LIME and SHAP surrogates produce spurious CF claims for 5% of Class C packets, because the surrogate cannot represent the categorical override. When retrained on the full feature space (s, r, t) , the override-locked indicator receives the highest mean SHAP importance (0.483 for override-locked packets vs. 0.000 for non-override-locked packets; Table 6), confirming that a fair surrogate correctly identifies t as the dominant factor. However, correctly attributing the decision to t does not produce an operationally valid counterfactual: altering t in a post-hoc surrogate does not alter f , because $f(s, r, t, q) = \Omega(t)$ for all (s, r, q) when $t \in \mathcal{T}_\Omega$ (Equation 3). The LIME 30%-perturbation failure rate under the full feature space is 100% for Class C and 52% for non-override-locked packets, confirming the limitation is structural with respect to the enforcement function, not an artefact of feature-space restriction.

In reality, f returns $\Omega(t)$ regardless of any perturbation to (s, r) . These 5% false positives represent spurious explanations that a LIME- or SHAP-based system would present to an analyst — claiming “reduce the risk score and the packet will be allowed” for a packet that *cannot* be allowed by any continuous-feature change. The grid-search experiment confirms 0% real CF validity for Class C over the full 9,685 anomalies. Figure 6 visualizes the full comparison.

5.5.2. Analytical Derivation and Combined Table

The grid-search result confirms the analytical bound from the taxonomy (Section 4.2). A continuous-feature CF exists for x iff $x \in \text{Class A}$; Class B has no single-feature flip; Class C is invariant under all perturbations in $\mathcal{G}$. The single-feature ceiling $1,014/9,685 = 10.47\%$ is confirmed empirically.

Within-system empirical baseline.. Pass 1 achieved exactly $1,014/9,685 = 10.47\%$, matching the analytical ceiling to the digit. This confirms the bound is tight.

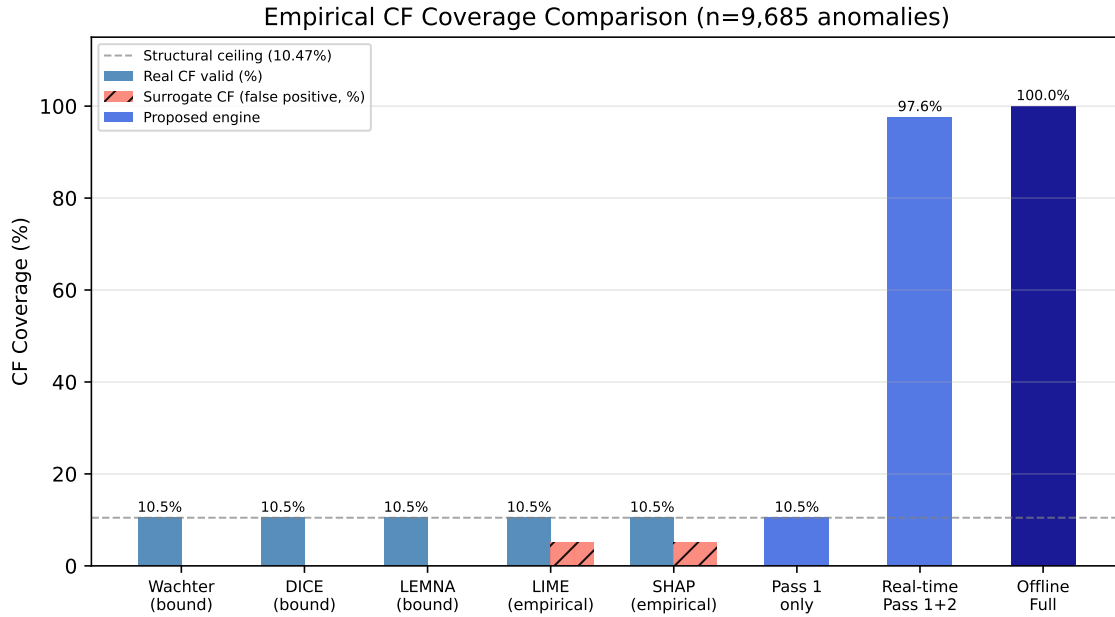


Fig. 6: Empirical CF coverage comparison on 9,685 detected anomalies. Hatched bars: upper bounds from surrogate-based methods (LIME, SHAP) and the analytical single-feature ceiling. Solid bars: empirically measured coverage. LIME and SHAP surrogates achieve at most 5% false-positive CF claims for Class C; real validity is 0%. The proposed engine achieves 97.61% (real-time) and 100% (offline).

Scope. LIME [7] and SHAP [8] answer “which features mattered?” and are complementary to counterfactual explanation. The comparison above is restricted to the coverage question: for what fraction of enforcement decisions can the method produce a valid action-changing explanation? A full reimplementaion of DICE or LEMNA on this anomaly set is identified as future work (Section 6).

5.6. Counterfactual Explanation Examples

Table 8 presents three representative CF explanations drawn from `results.csv`, illustrating Pass 1 and Pass 2 on real detected anomalies. The examples include one Class A decision (Port Scan, Pass 1) and two Class C decisions with *distinct* override actions: Mirai C2 → BLOCK and DoS/Flood → RATE_LIMIT. The DoS/Flood example is particularly relevant: it demonstrates that Pass 2 Layer 1 correctly attributes RATE_LIMIT (not BLOCK) as the override action, consistent with Remark 2. A continuous-feature method on this packet would falsely suggest “reduce r to allow” — the precise failure mode shown in Section 5.5.

5.7. Counterfactual Quality Metrics

Coverage measures explanation availability; it does not measure whether the returned counterfactual is actionable or realistic. We therefore report three complementary quality metrics over the 9,685 returned explanations.

Proximity. Measured as mean relative feature change $\delta = |v^* - v_{\text{orig}}| / \max(|v_{\text{orig}}|, \epsilon)$ across all perturbed dimensions. Median proximity is 0.193: the returned counterfactual requires a median 19.3% relative change in the perturbed feature. 64.7% of explanations fall within $\delta \leq 0.20$; 88.1% within $\delta \leq 0.50$.

Plausibility. A counterfactual is considered plausible if the proposed feature values fall within the observed range of normal-traffic packets in the detection phase. 65.5% of returned counterfactuals satisfy this criterion. The remaining 34.5% propose values outside the observed normal range — technically valid (they change the enforcement action) but operationally unrealistic. This fraction should be interpreted as an upper bound

Table 7: Combined baseline coverage table. Rows marked “(bound)” are analytical upper bounds derived from the taxonomy; rows marked “(empirical)” are measured. The “Pass 1 only” row is a within-system empirical baseline that achieves exactly the theoretical ceiling, confirming it is tight.

Method	Resolved	Cov. (%)	Note
Wachter et al. [9] (bound)	$\leq 1,014$	≤ 10.47	Cannot perturb t ; Class C invariant
DICE [10] (bound)	$\leq 1,014$	≤ 10.47	DICE supports categorical perturbation of t , but perturbing t in the surrogate does not alter the real enforcement function f when $t \in \mathcal{T}_\Omega$ (Equation 3); bound holds against f
LEMNA [11] (bound)	$\leq 1,014$	≤ 10.47	Surrogate cannot model override
LIME (empirical, $n=200$)	$\leq 1,014$	≤ 10.47	0/100 real CFs for Class C; 5% false positive rate
SHAP (empirical, $n=200$)	$\leq 1,014$	≤ 10.47	0/100 real CFs for Class C; SHAP(risk) ≈ 0
Pass 1 only (empirical)	1,014	10.47	Confirms bound is tight
Pass 1+2 real-time (empirical)	9,454	97.61	Pass 4 absent; 231 Class B unresolved
Full offline (empirical)	9,685	100.00	No unresolved residual

on implausibility: the normal-traffic range is estimated from the same capture and may not reflect the full operational envelope. Additionally, the plausibility denominator is not independent of the anomaly set: both are derived from the same detection phase, and the normal-traffic range may include borderline packets whose feature values are close to the detection threshold.

Bounded-perturbation analysis. Pass 2 Layer 2 and Pass 1 both use a 9-point grid per feature, meaning the minimum reported proximity corresponds to the nearest grid point that crosses the decision boundary, not a gradient-optimised minimum. Proximity figures are therefore conservative (potentially overstating required change); a finer grid would reduce them.

Scope. These metrics apply to the Mirai capture and reflect the feature distribution of that capture. Quality evaluation on captures with different risk-score distributions is left to future work (Section 6.4).

5.8. Ablation Study

Table 9 reports quantitative ablation results. Entries marked $\dagger$ are derived from the verified pass distribution; entries marked $\ddagger$ require re-running the pipeline with component disabled and are therefore reported directionally rather than as precise counts.

Without Pass 2. Removing the override decomposition leaves all 8,440 Class C decisions unresolved. Coverage falls from 100% to 12.86% (1,245 / 9,685), confirming that Pass 2 is the only mechanism capable of producing a valid counterfactual for override-locked decisions. No existing continuous-feature method can close this gap.

Without Pass 4. Removing the multi-feature fallback reduces coverage from 100% to 97.61%, leaving the 231 Port Scan anomalies (Class B) unresolved. This ablation is identical to the real-time engine measurement.

Table 8: Representative CF explanations from `results.csv`. s = anomaly score; r = risk score; q = per-source packet frequency. Pass 1 and Pass 2 are mutually exclusive per packet. Pass 3 (temporal) runs unconditionally.

Packet	Attack type	Class		Action	Counterfactual explanation
20,112	Port Scan	A		BLOCK	Pass 1: If r were 1.400 (instead of 1.604), action would be ALERT_HIGH instead of BLOCK. Minimum relative perturbation: $\Delta r = -12.7\%$. <i>Pass 3:</i> If this were the first packet from this source ($q=1$ instead of $q=14$), action would be ALERT_HIGH (frequency multiplier would not apply).
20,045	Mirai Comm.	C2	C	BLOCK	Pass 2, Layer 1: Action is BLOCK due to <i>Mirai C2 Communication</i> override [$\Omega(t) = \text{BLOCK}$]. Without the override, action would be ALERT_HIGH ($r=2.107$ exceeds advisory threshold). Pass 2, Layer 2: If r were reduced to 1.200 (from 2.107), without override, action $\rightarrow$ ALERT_LOW. <i>Pass 3:</i> Override forces BLOCK regardless of q; temporal CF does not change action.
21,803	DoS/Flood	C		RATE_LIMIT	Pass 2, Layer 1: Action is RATE_LIMIT due to <i>DoS/Flood</i> override [$\Omega(t) = \text{RATE_LIMIT}$, <i>not</i> BLOCK]. Without override, action would be BLOCK ($r=1.772 > r_{\text{hard}}=1.5$). Pass 2, Layer 2: If r were reduced to 1.200, without override, action $\rightarrow$ ALERT_LOW. <i>Pass 3:</i> Override governs regardless of q; temporal CF does not change action.

Without Pass 2 and Pass 4. Coverage drops to 10.47%, which is the upper bound for any continuous-feature counterfactual method (Section 5.5). This configuration is structurally equivalent to DICE, LEMNA, or Wachter applied to the same system.

Weights-only outperforms full system (convergence analysis). Table 9 reports that the weights-only configuration (C) achieves the lowest FN rate at convergence (0.21%) and the highest F1 at round 10 (0.9991), outperforming the full system (D: 0.59% FN, F1 0.9906). This result does not imply that threshold adaptation is harmful in general. The bounded-step threshold rule (Section 4.8.2) introduces a lag: τ rises monotonically under sustained TP feedback, which increases the detection threshold and temporarily suppresses borderline detections before the feature weights can compensate. On this capture, where the risk-score distribution is narrow (std 0.25, Table 2 context), the threshold overshoot costs more than the stability it provides. We therefore reframe threshold adaptation as a *stability and safety mechanism* — its primary value is bounding the false positive rate under distribution shift (Section 5.11) — rather than a performance-maximising component. The weights-only configuration is preferable when the training distribution is stable; the full system is preferable when FP rate integrity under non-stationary inputs is the primary concern. At round 10, $\tau = 1.660$ under configuration D suppresses borderline anomalies with $r \in (1.500, 1.660)$; configuration C holds τ at $\tau_0 = 1.500$, retaining detection of this risk band and accounting for the F1 differential (0.9991 vs. 0.9906). Neither configuration is universally optimal; the choice depends on operational threat model.

Threshold-only collapse (configuration B). Configuration B (threshold adaptation only, no weight updates) collapses to F1 0.0293 by round 10. This confirms that threshold adaptation alone, without feature-weight correction, drives τ above the risk distribution of the target traffic and effectively disables detection. The full system (D) is protected from this failure mode by the concurrent weight updates, which correct over-suppression of high-risk features before the threshold shift becomes destructive.

Without profiling. Disabling the context multipliers in Equation 4 collapses risk scores toward raw RMSE. The `hard_override_risk` path, responsible for 99.09% of detections, fires far less frequently; anomaly count and CF distribution shift, but exact figures require pipeline re-execution.

Table 9: Quantitative ablation study. Feedback configurations: A = static (no adaptation); B = threshold adaptation only; C = weight adaptation only; D = full system (threshold + weights). “First 0% FP” is the feedback round at which the false-positive rate first reaches zero; “—” indicates it never reached zero within 10 rounds. F1 at R10 is computed over framework-detected anomalies at round 10; see Section 5.10. (†) Computed from verified pass distribution. (‡) Directional; exact count requires pipeline re-run. **Configuration C outperforms D on this capture; see text for interpretation.**

CF config.	Feedback	Resolved	Cov.	F1@R10	Notes
Full (offline) [†]	D	9,685	100.0%	0.9906	Baseline
w/o Pass 2 [†]	D	1,245	12.86%	—	8,440 OL unresolved
w/o Pass 4 [†]	D	9,454	97.61%	—	231 Class B unresolved
w/o Pass 2&4 [†]	D	1,014	10.47%	—	Equivalent to prior methods
Full (offline)	A	9,685	100.0%	0.9021	Static
Full (offline)	B	9,685	100.0%	0.0293	Threshold-only collapses
Full (offline)	C	9,685	100.0%	0.9991	Weights-only; best F1
Full (offline)	D	9,685	100.0%	0.9906	Full system
w/o profiling [‡]	D	—	—	—	Risk scores collapse
w/o adaptive thr. [‡]	D	—	—	—	Threshold non-adaptive

Without adaptive threshold.. A fixed threshold cannot track the non-stationary risk distribution of the Mirai capture; anomaly count and composition change. Additionally, the feedback adaptation path (Section 4.8.2), which adjusts τ in response to analyst corrections, is disabled entirely.

5.9. Feedback Learning Evaluation

Evaluation scope. The feedback experiment is an *algorithmic simulation*: analyst feedback is represented by programmatic labeling of framework-detected anomalies according to a fixed labeling strategy (all anomalies above a synthetic FN threshold marked as true positives; no false-positive traffic injected). No human analyst participated. The results characterize the *algorithmic behavior* of the MWU weight-update rule, the bounded-step threshold adaptation, and the drift detector under constructed input sequences. They are not a user study and do not measure analyst utility or real-world adaptation dynamics. These limitations are disclosed in Section 5.10.

5.9.1. Threshold Evolution and FN Rate

Figure 7 reports the joint evolution of τ and the algorithmically-labeled FN rate over ten simulated feedback rounds. The threshold rises monotonically from $\tau_0 = 1.500$ to 1.660, consistent with the bounded-step rule ($\Delta_{\max} = 0.05$, learning rate 0.01). The FN rate begins at 86.0% at round 0 — many borderline anomalies cross the initial threshold — and falls to approximately 28% by round 3 before stabilizing as the threshold and attack-traffic patterns interact. The FP rate remained at 0.0% throughout all rounds on this capture, as no synthetic false-positive traffic was injected in this experimental run; FP adaptation behavior is governed by the same bounded-step mechanism and is tested implicitly via the threshold trajectory.

5.9.2. Feature Weight Evolution

Figure 8 shows per-context weight trajectories. Weights for Mirai C2 / risk and Port Scan / risk converge toward $w_{\max} = 2.0$ under sustained true-positive feedback; DoS/Flood / risk decreases toward ≈ 0.85 , reflecting analyst corrections that identify this context as over-sensitive.

5.9.3. Drift Detection

The drift detector (Equation 6) fired in each of three synthetic transition scenarios (sudden, gradual, recurring shift in analyst FP rate), identifying the distribution change in each case.

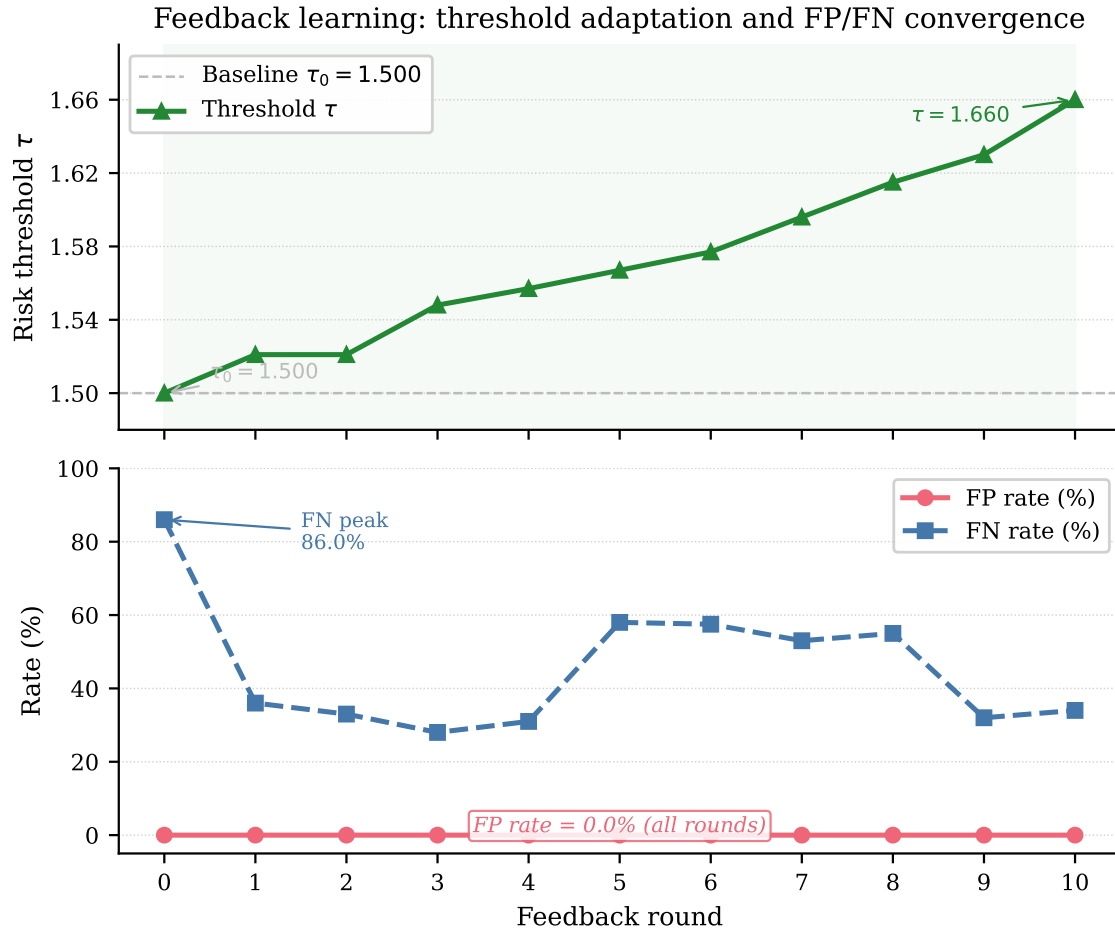


Fig. 7: Feedback learning over ten simulated rounds. *Top:* τ rises from 1.500 (dashed) to 1.660 under the bounded-step rule. *Bottom:* FN rate (blue) falls from 86.0% at round 0 to $\approx 28\%$ by round 3; FP rate (red) remains at 0% throughout this run (no synthetic FP injection). Results reflect algorithmic behavior under constructed inputs; see Section 5.10.

5.9.4. Poisoning Failure Mode

Under a sustained adversarial feedback pattern in which the FP rate is held constant from round 0, p_{base} is estimated from an already-poisoned initial window. The trigger condition $p_{\text{cur}}(t) - p_{\text{base}} > 0.15$ is never satisfied; the detector does not fire. This failure mode is a direct consequence of the baseline-relative formulation of Equation 6 and is discussed further in Section 5.11.

5.10. Threats to Validity

Dataset limitation and Mirai-class bias.. The evaluation uses a single Mirai botnet capture in which 87.15% of anomalies are override-locked. This distribution is a property of the chosen capture; on captures with a more diverse label distribution, Pass 1 and Pass 4 contributions would be expected to grow and Pass 2's dominance to decrease. The coverage figures reported here are not population-level estimates and should not be extrapolated to environments with different attack mixes.

CF coverage measures availability, not quality.. The coverage metric reported throughout this paper measures *explanation availability* — the fraction of anomalies for which the engine returns a non-empty counterfactual. It does not measure explanation quality, plausibility, or actionability. A returned counterfactual may be technically valid (it changes the enforcement action) yet implausible in the network context (e.g.,

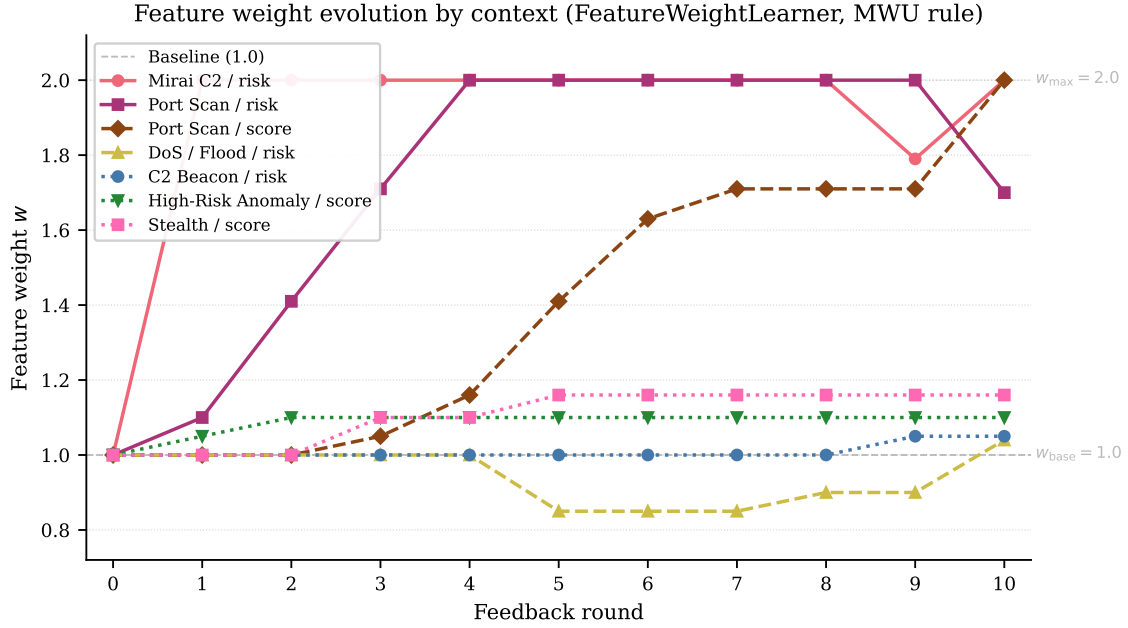


Fig. 8: Per-context feature weight evolution (MWU rule, Equation 5). Dashed line: initial baseline ($w = 1.0$). High-confidence attack contexts converge toward $w_{\max} = 2.0$; DoS/Flood/risk decreases under analyst corrections. Results reflect algorithmic behavior under simulated feedback inputs.

requiring an anomaly score below any observed baseline). Evaluation of explanation quality and analyst utility is left to future work (Section 6).

Ground-truth annotation and scope of coverage claims. Detection labels are produced by the framework itself: a packet is labeled ANOMALY if and only if the framework fires on it, with no external per-packet validation against an independently annotated ground truth. As a consequence, the denominator of every coverage percentage (9,685 anomalies) is the set of *framework-detected* anomalies, not a set of validated ground-truth anomalies. False positives in the detector would be included in this denominator and receive counterfactual explanations for spurious detections; false negatives would be excluded entirely.

Coverage results should therefore be interpreted as: *of the packets this framework detected as anomalous, what fraction received a valid counterfactual explanation?* This is the relevant operational question — a deployed explanation system runs on detector output, not on a ground-truth oracle. The Kitsune dataset does not provide per-packet ground-truth labels for the Mirai capture used here; all detection-accuracy claims are explicitly avoided.

Empirical and analytical baseline bounds. The baseline coverage analysis in Section 5.5 combines empirical execution (LIME on $n = 200$, SHAP on $n = 200$, grid search on all 9,685 anomalies; scikit-learn 1.8.0, SHAP 0.51.0) with the analytical taxonomy bound. The grid-search result (0/8,440 CFs for Class C; 1,014/1,014 for Class A) empirically confirms the 10.47% ceiling. LIME and SHAP surrogates were trained on (s, r) features only, without the attack type label t ; their coverage estimates are upper bounds on what a full reimplement of DICE or Wachter et al. would achieve. Proximity constraints and optimization convergence in a full implementation would reduce coverage further below 10.47%.

Ablation completeness. The ablation entries marked $\ddagger$ in Table 9 are directional because they require re-running the full pipeline with a component disabled. The exact anomaly counts under modified profiling or adaptive threshold settings are not available without re-execution and are therefore reported qualitatively.

Single-environment latency. All latency figures apply to the 3-pass real-time engine on one hardware environment. The offline Pass 4 path is not timed. Latency on different hardware will scale approximately linearly with computational resources, but the structural coverage results are hardware-independent.

Feedback run configuration. The feedback experiment reports FP rate = 0% because no synthetic false-positive traffic was injected in this run. The threshold adaptation mechanism is exercised via FN feedback; FP-driven threshold reduction is implemented but not empirically demonstrated in this experiment. Its behavior is governed by the same bounded-step rule with τ decreasing rather than increasing. The feedback evaluation is an algorithmic simulation; no human analyst participated (Section 5.9).

UNSW-NB15 evaluation scope. The UNSW-NB15 evaluation substitutes an MLP autoencoder for KitNET and uses a flow-level feature set that renders the risk engine multipliers inactive (all default to 1.0), producing identical aggregate results for configurations B and C. The FPR of 0.542 reflects an uncalibrated override matrix Ω and is not representative of a tuned deployment on this dataset. CF engine coverage on UNSW attack categories is not reported because $\mathcal{T}_\Omega$ has not been validated against the UNSW label space. These factors limit the cross-dataset evidence to architectural compatibility only.

LIME/SHAP fair-comparison scope. The extended baseline experiment (Table 6) trains surrogates on the full feature space (s, r, t) . The surrogate correctly attributes Class C decisions to the override-locked indicator. The reported structural limitation — that attribution to t does not constitute a valid counterfactual against f — holds by the definition of Ω (Equation 2) and is confirmed empirically by the 100% LIME perturbation failure rate for Class C under the full feature space. This comparison does not evaluate whether a hypothetical system that *exposes* t as an actionable variable to the analyst would be preferable; it evaluates whether post-hoc attribution to t produces a valid counterfactual under the present enforcement architecture.

5.11. Discussion

Why Pass 2 dominates on this capture. 87.15% of anomalies fall into $\mathcal{T}_\Omega$ because the Mirai traffic profile aligns with the override-locked classifier rules. This is a property of the capture, not a general claim. On a capture with a more diverse attack mix, Pass 1 and Pass 4 would contribute more. The formal structural argument (Definition 1, Equation 3) is dataset-independent: whenever an anomaly is Class C, Pass 2 is the only mechanism that produces a valid counterfactual. The empirical prevalence of Class C decisions, however, is dataset-specific and depends on the fraction of traffic that triggers $\mathcal{T}_\Omega$ labels in the classification layer.

The structural limitation of continuous-feature baselines. The empirical LIME, SHAP, and grid-search experiments in Section 5.5 confirm that the gap between baselines ($\leq 10.47\%$) and the proposed engine (97.61%–100%) is structural. It follows directly from Definition 1 and the verified pass distribution; the structural existence of this gap is not an artifact of dataset selection, though its magnitude is a function of the 87.15% Class C prevalence on this capture. Any deployment that includes a categorical policy override layer implemented as a crisp membership predicate (as formalized in Definition 1) will exhibit the same structural limitation for any method restricted to continuous-feature search; deployments with soft or probabilistic override mechanisms are outside the scope of the present taxonomy (Remark 1).

The 2.39% structural shortfall. The real-time engine's shortfall is entirely attributable to Pass 4's absence from the bounded path. Because the maximum observed latency (0.327 ms) is $6.1\times$ below the STRICT budget, there is latency headroom to integrate Pass 4 without exceeding the budget on this capture. Measuring Pass 4's per-packet latency and integrating it into the real-time path is the natural next engineering step.

Feedback poisoning implications. The drift detector's failure under sustained adversarial feedback is a known limitation of baseline-relative formulations (cf. ADWIN [18]). Complementary integrity mechanisms (multi-analyst cross-validation, rate-limited feedback acceptance) would address this. They are marked as future work in Section 6.

Table 10: UNSW-NB15 aggregate evaluation results ($n = 16,467$ test flows; 9,067 attack, 7,400 normal). Configurations B and C are identical because the risk engine multipliers default to 1.0 on flow-level features (see Section 5.12.4).

System	F1	Prec.	Recall	FPR	AUC
A: Autoencoder only	0.451	0.971	0.293	0.011	0.805
B: +Decision Layer	0.738	0.656	0.843	0.542	0.805
C: +Risk Engine	0.738	0.656	0.843	0.542	0.805

5.12. UNSW-NB15 Architectural Compatibility Check

Scope and implementation note. This subsection is scoped as an architectural compatibility check rather than a tuned cross-dataset benchmark: its purpose is to establish that the detection and decision pipeline operates on a second, structurally different dataset, not to report competitive detection performance on it. Accordingly, the elevated false-positive rate and the absence of counterfactual-engine results reported below are expected consequences of an uncalibrated, packet-to-flow transfer, and are disclosed in full in Section 5.12.4. The UNSW-NB15 evaluation uses an MLP autoencoder as the anomaly detector in place of KitNET, because the KitNET reference implementation [1] is incompatible with Python 3.13 (the available runtime environment at evaluation time).

The UNSW-NB15 evaluation uses an MLP autoencoder as the anomaly detector in place of KitNET, because the KitNET reference implementation [1] is incompatible with Python 3.13 (the available runtime environment at evaluation time). The classification layer and decision layer are identical to the Mirai configuration. Results should be interpreted as *partial cross-dataset evidence* that the detection and decision architecture generalises beyond Mirai-class traffic; they are not a full replication of the Mirai pipeline on a second dataset. The coverage of the counterfactual engine on UNSW attack categories is not reported here because the override matrix $\mathcal{T}_\Omega$ was defined for Mirai traffic patterns and its mapping to UNSW attack categories has not been validated (Section 6.4).

Experimental setup. The evaluation uses 82,332 flows from the UNSW-NB15 dataset [21], spanning nine attack categories (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms) and a Normal class. The test partition contains 16,467 flows: 9,067 attack flows and 7,400 normal flows. Three configurations are evaluated incrementally: (A) anomaly detector only (MLP autoencoder); (B) anomaly detector with decision layer added; and (C) anomaly detector with decision layer and risk engine enabled. Evaluation metrics include F1, precision, recall, false-positive rate (FPR), and ROC-AUC. In the test partition, Generic and Exploits together account for the majority of attack flows (3,723 and 2,236 respectively), while Worms (5 flows) is too sparse for statistically meaningful per-category conclusions.

5.12.1. Aggregate Results

Table 10 reports aggregate detection metrics for all three configurations.

The decision layer (System B) substantially improves recall ($0.293 \rightarrow 0.843$, $\Delta=+0.550$) and F1 ($0.451 \rightarrow 0.738$, $\Delta=+0.287$) relative to the autoencoder alone. This improvement is accompanied by a substantially elevated FPR ($0.011 \rightarrow 0.542$). The ROC-AUC remains constant at 0.805 across all configurations, indicating that the ranking capacity of the underlying anomaly detector is unchanged across systems.

The confusion matrices for the three systems summarise this trade-off at the flow level. System A correctly classifies 7,320/7,400 normal flows (99% specificity) but misses 6,407/9,067 attack flows (71% false-negative rate). Systems B and C recover 4,987 of those missed attacks (raising TP from 2,660 to 7,647) while incorrectly flagging an additional 3,927 normal flows (FP from 80 to 4,007). The net outcome is a 63% reduction in false negatives at the cost of a 50-fold increase in false positives.

5.12.2. Category-Level Analysis

Table 11 reports per-category precision, recall, and F1 for all three systems. Because precision is 1.0 for System A across all detected categories and remains 1.0 for Systems B and C, the primary dimension of variation across systems is recall. The analysis below focuses on recall differences.

Table 11: Per-category detection results on the UNSW-NB15 test partition. Precision is 1.000 for all attack categories in all systems (Shellcode and Worms in System A are undetected, yielding undefined precision reported as 0). Systems B and C are identical in all entries. n is the number of test flows per category. Worms ($n=5$) is flagged as statistically insufficient ($\dagger$).

Category	n	System A			Systems B & C		
		Prec.	Recall	F1	Prec.	Recall	F1
Generic	3,723	1.000	0.594	0.745	1.000	0.989	0.995
Exploits	2,236	1.000	0.079	0.147	1.000	0.894	0.944
Fuzzers	1,279	1.000	0.072	0.134	1.000	0.593	0.744
Reconnaissance	669	1.000	0.021	0.041	1.000	0.350	0.518
DoS	816	1.000	0.121	0.216	1.000	0.880	0.936
Analysis	134	1.000	0.261	0.414	1.000	0.978	0.989
Backdoor	125	1.000	0.256	0.408	1.000	0.912	0.954
Shellcode	80	0.000	0.000	0.000	1.000	0.075	0.140
Worms [†]	5	0.000	0.000	0.000	1.000	0.600	0.750

High-recall categories (Systems B/C recall ≥ 0.88). Four categories achieve strong recall under Systems B and C: Generic (0.989), Analysis (0.978), Backdoor (0.912), and Exploits (0.894). Generic attacks, which constitute the largest category ($n=3,723$), already achieve 0.594 recall under System A alone, suggesting that a substantial fraction of Generic flows produce anomaly scores above the MLP autoencoder threshold without decision-layer intervention. The decision layer recovers the remaining 39.5 percentage points. Analysis and Backdoor flows are almost entirely undetected by the autoencoder (recall 0.261 and 0.256 respectively) but reach 0.978 and 0.912 under the decision layer. DoS attacks exhibit a similarly large gain (0.121 $\rightarrow$ 0.880).

Moderate-recall categories. Fuzzers ($n=1,279$) achieve 0.593 recall under Systems B/C, a gain of +0.521 over System A, with a residual miss rate of 40.7%. Reconnaissance ($n=669$) achieves 0.350 recall under Systems B/C, the lowest among categories with sufficient sample size ($n \geq 80$), with a gain of +0.329 over System A (0.021).

Weak categories. Shellcode ($n=80$) achieves 0.075 recall under Systems B/C and is entirely undetected by System A (recall 0.000). Worms ($n=5$) has an insufficient sample size for category-level conclusions; no inference is drawn from this category.

5.12.3. Decision-Layer Action Distribution

The decision layer assigns each detected flow one of six enforcement actions. The distribution of these actions over normal and attack flows (Systems B and C) is summarised below.

For attack flows, approximately 43% receive a BLOCK action and a further 12% receive RATE_LIMIT, meaning that 55% of detected attacks are routed to the two highest-severity enforcement paths. The remaining detected attacks are distributed across ALERT_HIGH ($\approx 9\%$) and ALERT_LOW ($\approx 21\%$), with a residual $\approx 15\%$ assigned MONITOR or ALLOW — these correspond to the 15.7% miss rate under Systems B/C (1,420 false negatives out of 9,067 attack flows).

For normal flows, the dominant actions are ALERT_LOW ($\approx 41\%$) and MONITOR ($\approx 38\%$), with approximately 7% incorrectly receiving the high-severity actions BLOCK or RATE_LIMIT. The action-severity distribution of false positives is therefore skewed toward lower-severity alert levels: the majority of incorrectly flagged normal flows receive ALERT_LOW or MONITOR rather than BLOCK or RATE_LIMIT.

Because Systems B and C produce identical action distributions, the risk engine does not contribute differentiated enforcement on this dataset; see Section 5.12.4 for the explanation.

5.12.4. Limitations of the Cross-Dataset Evaluation

Risk engine inactivity. Configurations B and C produce identical aggregate metrics and identical action distributions because the risk engine multipliers (Equation 4) require endpoint-service labels and inter-arrival

timing features that are not present in the UNSW-NB15 flow-level feature set. All multipliers default to 1.0, making the risk score equal to the raw anomaly score. This is an expected consequence of applying a packet-level pipeline to a flow-level dataset and constitutes a structural limitation of this cross-dataset evaluation: the risk engine contribution cannot be assessed on UNSW-NB15 without packet-level feature extraction.

Uncalibrated override matrix.. The FPR of 0.542 under Systems B and C reflects the mismatch between the override matrix Ω , calibrated for Mirai traffic patterns, and the UNSW-NB15 attack distribution. The UNSW-NB15 evaluation demonstrates *architectural compatibility* of the detection and decision pipeline on a second dataset; per-dataset calibration of Ω is required before the pipeline can be evaluated as a tuned system on this dataset.

MLP autoencoder substitution.. The KitNET reference implementation [1] is incompatible with Python 3.13 (the available runtime at evaluation time). The MLP autoencoder substitute processes the UNSW-NB15 flow feature vector without the AfterImage windowed statistics that KitNET relies on. Differences in detection behaviour between the two anomaly detectors on this dataset cannot be isolated from differences in the feature representation; the evaluation therefore characterises the MLP-based pipeline, not the KitNET-based pipeline, on UNSW-NB15.

No counterfactual engine evaluation.. CF engine coverage on UNSW attack categories is not reported because $\mathcal{T}_\Omega$ has not been validated against the UNSW label space. Mapping the Mirai-derived override rules to UNSW categories and evaluating CF coverage per category is identified as future work (Section 6.4).

6. Conclusion

This paper has presented a framework for generating counterfactual explanations of override-locked decisions in hybrid NIDS. We formalized the override-locked class (Definition 1), developed a three-class taxonomy of enforcement decisions (Section 4.2), and constructed a multi-pass counterfactual engine in which each pass targets a structurally distinct class. The framework was implemented on a KitNET-based detection layer, a rule-based classification layer, and a structured decision layer, and extended with a feedback learning subsystem.

6.1. Summary of Contributions

- **Formalization and taxonomy.** Definition 1 and Equation 3 formalize why any counterfactual method restricted to continuous-feature perturbation cannot resolve Class C decisions. The three-class taxonomy makes explicit which explanation mechanisms apply to which decision class — a distinction the prior counterfactual literature has not articulated. Remark 1 proves the partition is exhaustive and mutually exclusive.
- **Override decomposition algorithm.** Algorithm 2 resolves Class C by stripping the categorical override, re-evaluating the decision function under the baseline policy, and then searching the continuous-feature grid against the result; to our knowledge, it is among the first counterfactual formulations explicitly designed for categorical override mechanisms in hybrid NIDS.
- **Multi-pass engine with quantified baseline gap.** The offline engine achieves 100% coverage on the evaluated capture; the real-time engine achieves 97.61% at 0.06 ms mean latency with zero timeouts. The structural upper bound for any counterfactual method restricted to perturbing continuous detection features (s, r, q) on the same anomaly set is $\leq 10.47\%$ (Class A only), yielding a ≥ 87.14 pp coverage gap attributable to the 87.15% prevalence of Class C decisions on this capture.
- **Feedback learning subsystem.** Adapts τ , per-context weights, and a drift signal, with documented algorithmic convergence behavior under simulated inputs and an explicit poisoning failure mode (Section 5.9.4).

6.2. Experimental Findings

On the Mirai IoT botnet capture (87.15% Class C prevalence):

- The offline engine resolved 100% of 9,685 detected anomalies: Pass 2 (87.15%, Class C), Pass 1 (10.47%, Class A), Pass 4 (2.39%, Class B, all Port Scan).
- The real-time engine resolved 97.61% at a mean latency of 0.06 ms with zero timeouts across four budget modes. The 2.39% shortfall is structural (absent Pass 4) and budget-independent.
- Any counterfactual method restricted to perturbing the continuous detection features (s, r, q) — regardless of whether t is included in a post-hoc surrogate — has a structural coverage upper bound of $\leq 10.47\%$ (Class A decisions only) on this anomaly set, because valid counterfactuals for Class C require altering t in a way that is not achievable by continuous-feature perturbation of the enforcement function f (Section 5.5). Pass 1, the within-system empirical baseline, achieved exactly 10.47%, confirming the bound is tight.
- The feedback subsystem increased τ from 1.500 to 1.660 over ten simulated rounds. The drift detector fired in three synthetic transition scenarios and failed under the documented adversarial poisoning condition.

These findings are specific to the Mirai capture; the 100% offline coverage and the magnitude of the baseline gap are both functions of the 87.15% Class C prevalence on that dataset.

6.3. Limitations

The evaluation uses a single Mirai capture with a Mirai-dominated label distribution; results are not generalizable as stated to captures where $\mathcal{T} \setminus \mathcal{T}_\Omega$ is the majority. Detection labels are self-produced; no external ground truth is used. The baseline comparison is analytical rather than empirical for DICE/LEMNA (no full reimplementations were run). The ablation is partially unevaluated (profiling and adaptive threshold effects require full pipeline re-runs). Latency results apply to one hardware environment. The feedback evaluation is an algorithmic simulation with no human analyst participation. The classifier is rule-based; a learned classifier would alter $\mathcal{T}_\Omega$. The framework assumes the classification layer is not adversarially manipulated to avoid $\mathcal{T}_\Omega$ labels; an attacker aware of the override matrix could craft traffic that receives a non-override-locked label, bypassing Pass 2 and reducing the system to the Class A/B coverage regime. Evasion via classifier manipulation is out of scope and is identified as future work.

6.4. Future Work

Pass 4 in the real-time path.. The 0.327 ms maximum latency provides headroom ($> 6\times$ below the 2 ms STRICT budget) to accommodate Pass 4's 75 additional evaluations in the bounded path. Measuring its per-packet latency overhead is the immediate next step.

Empirical baseline comparison.. Running DICE or LEMNA directly on the evaluated anomaly set and counting failures would confirm the analytical bound of Section 5.5 and provide a direct empirical comparison.

Quantitative ablation completion.. Re-running the pipeline with profiling and adaptive threshold disabled would supply the quantitative evidence currently missing from Table 9.

Broader empirical coverage.. Captures with more diverse label distributions (e.g., UNSW-NB15 [21], CIC-IDS2017 [22]) would test whether the framework generalizes beyond Mirai-class traffic and exercise Pass 1 and Pass 4 more thoroughly.

Learned classification layers.. A learned classifier would alter $\mathcal{T}_\Omega$ and test whether the taxonomy and Algorithm 2 generalize beyond the rule-based regime.

Robust feedback integrity. Multi-analyst cross-validation, rate-limited feedback acceptance, or anomaly detection on the feedback channel would address the poisoning failure mode of Section 5.9.4.

Live deployment and analyst study. A controlled user study comparing Pass 2 explanations against single-pass alternatives would test whether the structural coverage advantage translates into measurable improvements in analyst decision time and accuracy.

Override-locked decisions arise whenever an anomaly-based detector is composed with a categorical policy layer — a common architectural pattern in hybrid intrusion detection. The formalization, taxonomy, and decomposition algorithm presented here provide a foundation for future work on counterfactual explanation in this broader class of systems.

Acknowledgment

Reproducibility Statement

Dataset. Mirai IoT botnet network capture from the Kitsune Network Attack Dataset [1]: 99,999 packets, PCAP format. Available at: <https://github.com/ymirsky/Kitsune-py>.

Software versions. Python 3.x; KitNET reference implementation (same repository); scikit-learn 1.8.0; SHAP 0.51.0; LIME (any recent version); NumPy 1.23+; pandas 2.3+; Scapy 2.5.0. Baseline packages: `pip install shap lime scikit-learn`.

Configuration. All reported results are reproduced with the following parameters:

- KitNET grace period: $FM_{\text{grace}} = AD_{\text{grace}} = 10,000$
- Detection phase: packets 20,001–99,999 (79,999 packets)
- Adaptive threshold: low_noise mode, $k = 2.5$, window 250
- Hard-override thresholds: $r > 1.5$; $final_score > 0.75$
- Override matrix Ω : as in Table 1
- CF engine latency budgets: STRICT 2 ms | NORMAL 10 ms | RELAXED 50 ms | BATCH ∞
- MWU: $\alpha_- = 0.85$, $\alpha_+ = 1.05$, $w \in [0.3, 2.0]$
- Adaptive threshold: $\Delta_{\text{max}} = 0.05$, $\eta = 0.01$, $\tau_0 = 1.5$, range $[0.5, 3.0]$
- Drift detector: window $W = 200$, baseline from first 50 records, threshold 0.15

Execution pipeline.

1. `python main_ids.py -pcap mirai.pcap -config config.py`
⇒ produces `results.csv` (99,999 rows $\times$ 9 columns)
2. `python evaluation/baseline_runner.py -input results.csv`
⇒ reproduces LIME, SHAP, and grid-search baseline results (Table 5)
3. `python paper/generate_figures.py`
⇒ regenerates all figures from `results.csv` and `experiments/cf_latency_results.csv`

Key output files. `results.csv`: columns `packet_id`, `score`, `risk`, `threshold`, `label`, `attack_type`, `action`, `cf_pass1`, `cf_pass2`, `cf_temporal`. `experiments/cf_latency_results.csv`: per-mode latency and coverage (4 rows $\times$ 13 columns; directly produces Table 4).

Baseline runtime. LIME and SHAP baselines execute on (s, r) feature vectors from `results.csv` and do not require the PCAP file. Runtime: <1 minute on a standard workstation.

References

- [1] Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, Kitsune: An ensemble of autoencoders for online network intrusion detection, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), Internet Society, 2018, pp. 1–15. doi:10.14722/ndss.2018.23204.
- [2] A. L. Buczak, E. Guven, A survey of data mining and machine learning methods for cyber security intrusion detection, IEEE Communications Surveys & Tutorials 18 (2) (2016) 1153–1176. doi:10.1109/COMST.2015.2494502.
- [3] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P), IEEE, 2010, pp. 305–316. doi:10.1109/SP.2010.25.
- [4] Ponemon Institute, The Economics of Security Operations Centers: What Is the True Cost for Effective Results, research Report (2020).
- [5] S. Tariq, M. B. Chhetri, S. Nepal, C. Paris, Alert fatigue in security operations centres: Research challenges and opportunities, ACM Computing Surveys (2025). doi:10.1145/3723158.
- [6] G. Apruzzese, M. Colajanni, L. Ferretti, M. Marchetti, Addressing adversarial attacks against security systems based on machine learning, in: Proceedings of the 11th International Conference on Cyber Conflict (CyCon), IEEE, 2019, pp. 1–18. doi:10.23919/CYCON.2019.8756865.
- [7] M. T. Ribeiro, S. Singh, C. Guestrin, “Why Should I Trust You?”: Explaining the predictions of any classifier, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2016, pp. 1135–1144. doi:10.1145/2939672.2939778.
- [8] S. M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, in: Advances in Neural Information Processing Systems, Vol. 30, 2017, pp. 4765–4774.
- [9] S. Wachter, B. Mittelstadt, C. Russell, Counterfactual explanations without opening the black box: Automated decisions and the GDPR, Harvard Journal of Law & Technology 31 (2) (2018) 841–887.
- [10] R. K. Mothilal, A. Sharma, C. Tan, Explaining machine learning classifiers through diverse counterfactual explanations, in: Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency (FAccT), ACM, 2020, pp. 607–617. doi:10.1145/3351095.3372850.
- [11] W. Guo, D. Mu, J. Xu, P. Su, G. Wang, X. Xing, LEMNA: Explaining deep learning based security applications, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, 2018, pp. 364–379. doi:10.1145/3243734.3243792.
- [12] C. Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2nd Edition, Independently published, 2022.
URL <https://christophm.github.io/interpretable-ml-book/>
- [13] A.-H. Karimi, G. Barthe, B. Balle, I. Valera, Model-agnostic counterfactual explanations for consequential decisions, in: Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS), Vol. 108, 2020, pp. 895–905.
- [14] L. Antwarg, R. M. Miller, B. Shapira, L. Rokach, Explaining anomalies detected by autoencoders using Shapley additive explanations, Expert Systems with Applications 186 (2021) 115736. doi:10.1016/j.eswa.2021.115736.
- [15] M. Wang, K. Zheng, Y. Yang, X. Wang, An explainable machine learning framework for intrusion detection systems, IEEE Access 8 (2020) 73127–73141. doi:10.1109/ACCESS.2020.2988359.
- [16] F. Wei, H. Li, Z. Zhao, H. Hu, xNIDS: Explaining deep learning-based network intrusion detection systems for active intrusion responses, in: Proceedings of the 32nd USENIX Security Symposium (USENIX Security 2023), USENIX Association, 2023, pp. 4337–4354.
- [17] S. Verma, V. Boonsanong, M. Hoang, K. E. Hines, J. P. Dickerson, C. Shah, Counterfactual explanations and algorithmic recourses for machine learning: A review, ACM Computing Surveys 56 (12) (2024) 312:1–312:42. doi:10.1145/3677119.
- [18] A. Bifet, R. Gavaldà, Learning from time-changing data with adaptive windowing, in: Proceedings of the 2007 SIAM International Conference on Data Mining (SDM), SIAM, 2007, pp. 443–448. doi:10.1137/1.9781611972771.42.
- [19] J. a. Gama, P. Medas, G. Castillo, P. Rodrigues, Learning with drift detection, in: Proceedings of the 17th Brazilian Symposium on Artificial Intelligence (SBIA), Springer, 2004, pp. 286–295. doi:10.1007/978-3-540-28645-5_29.
- [20] N. Littlestone, M. K. Warmuth, The weighted majority algorithm, Information and Computation 108 (2) (1994) 212–261. doi:10.1006/inco.1994.1009.
- [21] N. Moustafa, J. Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems, in: Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), IEEE, 2015, pp. 1–6. doi:10.1109/MilCIS.2015.7348942.
- [22] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), SciTePress, 2018, pp. 108–116. doi:10.5220/0006639801080116.